IMPLEMENTATION GUIDE · V1.0

Build the KSP Crime Analytics Platform

7-Day Sprint · Step-by-Step · Catalyst-Native · smolagents-Powered

A complete, day-by-day implementation guide for building the AI-Driven Crime Analytics & Visualization Platform from zero to demo-ready in seven days. Every step has a clear objective, the exact files to create, the commands to run, and the verification to confirm it worked. Follow this guide in order and you will have a deployable, jury-ready prototype on Day 7.

7-DAY SPRINT | CATALYST CLI | smolagents | 3 PIPELINES | 15 TOOLS | EVAL SET | DEMO-READY

7	45	3	50
DAYS	STEPS	PIPELINES	EVAL QUERIES

D1 FOUNDATION | D2 SMOLAGENTS | D3 ALERTS | D4 NETWORK | D5 PREDICTIVE | D6 LINEAGE | D7 DEMO

PREPARED FOR	DEPLOY ON	GOAL
Dev Team (2-3 ppl)	Zoho Catalyst	Win KSP Datathon 2026

How to Use This Guide

This guide takes you from zero to a deployable, jury-ready KSP Crime Analytics and Visualization Platform in seven days. It is organized by day, and each day is broken into numbered steps. Every step has five parts: a clear objective, the files to create or edit, the commands or code to run, a verification step to confirm it worked, and an estimated time. Follow the steps in order within each day; do not skip verification steps, because each step builds on the previous one and a silent failure on Day 2 will cost you hours on Day 5.

The guide assumes a 2-3 person team: one backend developer (owns Catalyst Functions, Data Store, pipelines), one frontend developer (owns Slate dashboard, visualizations), and one floating developer (owns integration, testing, demo prep). Where a step is owned by one role, it is marked [BE], [FE], or [INT]. Steps without a marker are collaborative. The guide also assumes you have the blueprint PDF and the smolagents addendum PDF open for reference; this guide tells you what to build, those documents tell you why.

Prerequisites

#	Prerequisite	How to Verify	Time
1	Zoho Catalyst account with Datathon credits claimed	Login at catalyst.zoho.com, see credits in billing	5 min
2	Catalyst CLI installed and authenticated	Run `catalyst --version` and `catalyst whoami`	10 min
3	Node.js 20+ and npm 10+ installed	Run `node --version` (need v20+)	5 min
4	Python 3.12 installed locally	Run `python3 --version` (need 3.10-3.12)	5 min
5	Git and GitHub account (public repo for submission)	Run `git --version` and `gh auth status`	5 min
6	Mapbox account + free API token	Sign up at mapbox.com, get a public token	5 min
7	VS Code or equivalent editor with Python + React extensions	Open VS Code, check extensions	5 min
8	Read the blueprint PDF and smolagents addendum PDF end-to-end	Both in /download/ folder	60 min
9	Team kickoff meeting: assign roles, set up shared Slack/Discord	All members present, roles agreed	30 min
10	GitHub repo created: ksp-crime-analytics-platform (public)	Repo URL noted, all members have push access	10 min

> **Iron rule**
>
> Do not start Day 1 until every prerequisite is verified. A team that starts building without the Catalyst CLI authenticated will lose 2 hours on Day 1 morning. Verify all 10 prerequisites the night before Day 1 begins.

Repository Structure

Set up this repository structure on Day 1 before writing any code. The structure mirrors the Catalyst deployment: each Catalyst Function lives in its own folder under functions/, the web client lives in web-client/, the synthetic data generator lives in scripts/, and the eval suite lives in tests/. This structure makes deployment scripted and reproducible, which matters when you re-deploy on Day 7.

```
ksp-crime-analytics-platform/<br/>├── README.md # Setup instructions, deploy
steps<br/>├── requirements.txt # Python deps (smolagents, zcatalyst-sdk, etc.)<br/>├──
package.json # Root npm workspace config<br/>├── catalyst.json # Catalyst project
config (CLI-managed)<br/>├── .env.example # Environment variable
template<br/>|<br/>├── functions/ # Catalyst Functions (Python 3.12)<br/>| ├──
orchestrator/ # smolagents CodeAgent + handler<br/>| | ├── handler.py<br/>| | ├──
router_agent.py # (from smolagents skeleton)<br/>| | └── requirements.txt<br/>| ├──
visualization/ # 5 viz pipeline tools<br/>| | ├── trend_query.py<br/>| | ├──
hotspot_kde.py<br/>| | ├── drill_down.py<br/>| | ├── emerging_trend.py<br/>| | └──
seasonal_decomp.py<br/>| ├── network/ # 5 network pipeline tools<br/>| | ├──
build_graph.py<br/>| | ├── detect_communities.py<br/>| | ├── repeat_offender.py<br/>|
| ├── mo_cluster.py<br/>| | └── centrality.py<br/>| ├── predictive/ # 5 predictive
pipeline tools<br/>| | ├── socio_correlation.py<br/>| | ├── risk_score.py<br/>| | ├──
forecast.py<br/>| | ├── anomaly_detect.py<br/>| | └── feature_importance.py<br/>| ├──
pdf_export/ # SmartBrowz PDF generation<br/>| | └── handler.py<br/>| └──
nightly_batch/ # Cron jobs (4 nightly)<br/>| ├── hotspot_recompute.py<br/>| ├──
forecast_refresh.py<br/>| ├── anomaly_scan.py<br/>| └── graph_refresh.py<br/>|<br/>├──
web-client/ # Slate React SPA<br/>| ├── package.json<br/>| ├── vite.config.ts<br/>|
├── src/<br/>| | ├── App.tsx<br/>| | ├── main.tsx<br/>| | ├── components/<br/>| | | ├──
Dashboard.tsx # SCRB strategic dashboard<br/>| | | ├── DistrictDrillDown.tsx # Map +
drill-down<br/>| | | ├── NetworkGraph.tsx # ECharts force-directed<br/>| | | ├──
PredictivePanel.tsx # Risk choropleth + forecast<br/>| | | ├── RepeatOffender.tsx #
Offender profile<br/>| | | ├── LineageViewer.tsx # "Why this?" modal<br/>| | | └──
KpiStrip.tsx<br/>| | ├── pages/<br/>| | | ├── Login.tsx<br/>| | | ├──
AnalystDashboard.tsx<br/>| | | └── SpDashboard.tsx<br/>| | ├── lib/<br/>| | | ├──
api.ts # Catalyst API client<br/>| | | ├── auth.ts # JWT handling<br/>| | | └──
mapbox.ts # Mapbox config<br/>| | └── styles/<br/>| | └── globals.css<br/>| └──
index.html<br/>|<br/>├── schema/ # Catalyst Data Store schema<br/>| ├── tables.sql #
14 table definitions<br/>| └── seed_indexes.sql # Indexes for
performance<br/>|<br/>├── scripts/ # Dev scripts (not deployed)<br/>| ├──
generate_synthetic_data.py # 50K FIRs generator<br/>| ├── load_data.py # Bulk import
to Data Store<br/>| ├── train_automl.py # Zia AutoML model training<br/>| └── deploy.sh
# Full deployment script<br/>|<br/>├── tests/ # Eval suite<br/>| ├── eval_queries.json
# 50 sample queries + expected<br/>| ├── run_eval.py # Eval runner<br/>| └──
benchmarks/ # Performance benchmarks<br/>|<br/>└── docs/ # Documentation<br/> ├──
blueprint.pdf # The analytics blueprint<br/> ├── smolagents_addendum.pdf # smolagents
integration<br/> └── api_contracts.md # API contracts (from blueprint)
```

Day 1 — Foundation (Catalyst, Schema, Data, Auth, Scaffold)

Day 1 lays the foundation that every subsequent day builds on. The goal is to have: a Catalyst project created, the 14-table Data Store schema live with 50,000 synthetic FIRs loaded, Authentication configured with the 4-role hierarchy, the Slate web client scaffolded with login working, and the GitHub repo structured per the layout above. Do not write any pipeline code today; if you finish early, spend the time reading the blueprint sections on the three pipelines so Day 2 starts with full context.

> **Day 1 success criteria**
>
> By end of Day 1: (1) `catalyst whoami` works, (2) `catalyst datastore:tables:list` shows 14 tables, (3) `catalyst datastore:row:select * from fir LIMIT 5` returns 5 rows, (4) `catalyst auth:users:list` shows test users in 4 roles, (5) `cd web-client && npm run dev` serves a login page that authenticates against Catalyst, (6) all code pushed to GitHub.

Step 1.1 — Create Catalyst Project [BE]

Objective: Initialize the Catalyst project that will host all backend services.

Commands:

```
# Install Catalyst CLI (if not done in prereqs)<br/>npm install -g
@zohocornerstone/cli<br/><br/># Authenticate<br/>catalyst login<br/><br/># Create
project<br/>catalyst project:create --name ksp-crime-analytics \<br/> --description
"KSP Crime Analytics & Visualization Platform" \<br/> --environment
development<br/><br/># Initialize in your repo<br/>cd
ksp-crime-analytics-platform<br/>catalyst init --project-id
<PROJECT_ID_FROM_ABOVE><br/><br/># Verify<br/>catalyst whoami<br/>catalyst
project:list
```

Verify: catalyst whoami prints your email and project ID. catalyst.json file exists in repo root.

Time: 20 minutes

Step 1.2 — Create Data Store Schema (14 Tables) [BE]

Objective: Define all 14 relational tables in Catalyst Data Store.

Files to create: schema/tables.sql

Commands: First write the schema file, then import it.

```
-- schema/tables.sql<br/>-- Run: catalyst datastore:bulk-import
schema/tables.sql<br/><br/>CREATE TABLE district (<br/> district_id BIGINT PRIMARY
KEY,<br/> district_name VARCHAR(100) NOT NULL,<br/> division VARCHAR(50),<br/>
population BIGINT,<br/> urbanization_rate DECIMAL(5,2),<br/> lat DECIMAL(10,7),<br/>
lng DECIMAL(10,7)<br/>);<br/><br/>CREATE TABLE police_station (<br/> ps_id BIGINT
PRIMARY KEY,<br/> ps_name VARCHAR(150) NOT NULL,<br/> district_id BIGINT NOT
NULL,<br/> range_name VARCHAR(50),<br/> lat DECIMAL(10,7),<br/> lng
DECIMAL(10,7),<br/> FOREIGN KEY (district_id) REFERENCES
district(district_id)<br/>);<br/><br/>CREATE TABLE crime_type (<br/> crime_type_id
```

```
BIGINT PRIMARY KEY,<br/> ipc_section VARCHAR(20) NOT NULL,<br/> category
VARCHAR(50),<br/> severity INT,<br/> description TEXT<br/>);<br/><br/>CREATE TABLE
modus_operandi (<br/> mo_id BIGINT PRIMARY KEY,<br/> mo_name VARCHAR(200) NOT
NULL,<br/> category VARCHAR(50),<br/> typical_crime_type_id BIGINT,<br/> FOREIGN KEY
(typical_crime_type_id) REFERENCES crime_type(crime_type_id)<br/>);<br/><br/>CREATE
TABLE accused (<br/> accused_id BIGINT PRIMARY KEY,<br/> name VARCHAR(200) NOT
NULL,<br/> alias VARCHAR(200),<br/> dob DATE,<br/> gender VARCHAR(10),<br/> address
TEXT,<br/> occupation VARCHAR(100),<br/> education VARCHAR(50),<br/>
socio_economic_status VARCHAR(30),<br/> first_offense_date DATE<br/>);<br/><br/>CREATE
TABLE victim (<br/> victim_id BIGINT PRIMARY KEY,<br/> name VARCHAR(200),<br/> dob
DATE,<br/> gender VARCHAR(10),<br/> address TEXT,<br/> occupation VARCHAR(100),<br/>
victim_type VARCHAR(30)<br/>);<br/><br/>CREATE TABLE fir (<br/> fir_id BIGINT PRIMARY
KEY,<br/> fir_number VARCHAR(50) NOT NULL,<br/> ps_id BIGINT NOT NULL,<br/>
crime_type_id BIGINT NOT NULL,<br/> fir_date DATE NOT NULL,<br/> fir_time TIME,<br/>
status VARCHAR(50),<br/> summary TEXT,<br/> lat DECIMAL(10,7),<br/> lng
DECIMAL(10,7),<br/> mo_description TEXT,<br/> FOREIGN KEY (ps_id) REFERENCES
police_station(ps_id),<br/> FOREIGN KEY (crime_type_id) REFERENCES
crime_type(crime_type_id)<br/>);<br/><br/>CREATE TABLE fir_accused (<br/> fir_id
BIGINT NOT NULL,<br/> accused_id BIGINT NOT NULL,<br/> role VARCHAR(50),<br/>
arrest_date DATE,<br/> bail_status VARCHAR(30),<br/> PRIMARY KEY (fir_id,
accused_id),<br/> FOREIGN KEY (fir_id) REFERENCES fir(fir_id),<br/> FOREIGN KEY
(accused_id) REFERENCES accused(accused_id)<br/>);<br/><br/>CREATE TABLE fir_victim
(<br/> fir_id BIGINT NOT NULL,<br/> victim_id BIGINT NOT NULL,<br/> injury_severity
VARCHAR(30),<br/> PRIMARY KEY (fir_id, victim_id)<br/>);<br/><br/>CREATE TABLE fir_mo
(<br/> fir_id BIGINT NOT NULL,<br/> mo_id BIGINT NOT NULL,<br/> PRIMARY KEY (fir_id,
mo_id)<br/>);<br/><br/>CREATE TABLE case_status (<br/> status_id BIGINT PRIMARY
KEY,<br/> fir_id BIGINT NOT NULL,<br/> status VARCHAR(50) NOT NULL,<br/> updated_at
TIMESTAMP NOT NULL,<br/> updated_by VARCHAR(100),<br/> FOREIGN KEY (fir_id) REFERENCES
fir(fir_id)<br/>);<br/><br/>CREATE TABLE offender_history (<br/> history_id BIGINT
PRIMARY KEY,<br/> accused_id BIGINT NOT NULL,<br/> prior_fir_id BIGINT NOT NULL,<br/>
prior_crime_type_id BIGINT,<br/> conviction_status VARCHAR(30),<br/> FOREIGN KEY
(accused_id) REFERENCES accused(accused_id),<br/> FOREIGN KEY (prior_fir_id)
REFERENCES fir(fir_id)<br/>);<br/><br/>CREATE TABLE financial_transaction (<br/>
txn_id BIGINT PRIMARY KEY,<br/> accused_id BIGINT NOT NULL,<br/> account_number
VARCHAR(30),<br/> amount DECIMAL(15,2),<br/> txn_date DATE,<br/> txn_type
VARCHAR(20),<br/> flagged_suspicious BOOLEAN DEFAULT FALSE,<br/> FOREIGN KEY
(accused_id) REFERENCES accused(accused_id)<br/>);<br/><br/>CREATE TABLE
socio_economic_indicator (<br/> indicator_id BIGINT PRIMARY KEY,<br/> district_id
BIGINT NOT NULL,<br/> year INT NOT NULL,<br/> literacy_rate DECIMAL(5,2),<br/>
unemployment_rate DECIMAL(5,2),<br/> migration_index DECIMAL(5,2),<br/>
urbanization_rate DECIMAL(5,2),<br/> gdp_per_capita DECIMAL(12,2),<br/> FOREIGN KEY
(district_id) REFERENCES district(district_id)<br/>);<br/><br/>-- Indexes for query
performance<br/>CREATE INDEX idx_fir_date_ps ON fir(fir_date, ps_id);<br/>CREATE INDEX
idx_fir_crime_type ON fir(crime_type_id, fir_date);<br/>CREATE INDEX
idx_fir_accused_accused ON fir_accused(accused_id);<br/>CREATE INDEX
idx_offender_history_accused ON offender_history(accused_id);<br/>CREATE INDEX
idx_fir_lat_lng ON fir(lat, lng);
```

Run: catalyst datastore:bulk-import schema/tables.sql

Verify: catalyst datastore:tables:list shows 14 tables. catalyst datastore:table:describe fir shows the fir schema.

Time: 45 minutes

Step 1.3 — Generate Synthetic Data [BE]

Objective: Generate 50,000 FIRs with realistic distributions, seasonal patterns, named gangs, and socio-economic correlations so the demo is compelling.

Files to create: scripts/generate_synthetic_data.py, scripts/load_data.py

Code (generate_synthetic_data.py):

```
#!/usr/bin/env python3<br/>"""Generate 50K synthetic FIRs for KSP demo with realistic
patterns."""<br/>import random<br/>import csv<br/>import json<br/>from datetime import
datetime, timedelta<br/>from pathlib import Path<br/><br/>random.seed(42) #
reproducible<br/><br/>DISTRICTS = [<br/> ("Bengaluru Urban", "Bengaluru", 9600000,
89.1, 12.9716, 77.5946),<br/> ("Bengaluru Rural", "Bengaluru", 990000, 58.2, 12.8467,
77.5167),<br/> ("Mysuru", "Mysuru", 3200000, 53.4, 12.2958, 76.6394),<br/>
("Hubli-Dharwad", "Dharwad", 1800000, 51.7, 15.3647, 75.1240),<br/> ("Mangaluru",
"Dakshina Kannada", 1900000, 64.3, 12.9141, 74.8560),<br/> # ... add 26 more Karnataka
districts to total 31<br/>]<br/><br/>CRIME_TYPES = [<br/> (1, "IPC 379", "Theft", 3,
"Theft of property"),<br/> (2, "IPC 392", "Robbery", 4, "Robbery with force"),<br/> (3,
"IPC 302", "Murder", 5, "Murder"),<br/> (4, "IPC 354", "Assault", 3, "Assault on
woman"),<br/> (5, "IPC 420", "Cheating", 2, "Cheating / fraud"),<br/> (6, "IPC 376",
"Rape", 5, "Sexual assault"),<br/> # ... add 114 more to total 120 IPC
sections<br/>]<br/><br/># Named gangs for persistent co-accused
relationships<br/>GANGS = [<br/> {"name": "Jayanagar Theft Ring", "home_district":
"Bengaluru Urban",<br/> "members": 8, "crime_type": 1, "active_from":
"2023-06-01"},<br/> {"name": "Mysuru Highway Robbers", "home_district": "Mysuru",<br/>
"members": 6, "crime_type": 2, "active_from": "2023-03-15"},<br/> # ... add 8-10 more
gangs<br/>]<br/><br/># Modus operandi taxonomy<br/>MO_TAXONOMY = [<br/> (1, "Forced
entry rear window", "burglary", 1),<br/> (2, "Chain snatching by bike-borne",
"snatching", 1),<br/> (3, "ATM skimming", "cybercrime", 5),<br/> # ... 85 total
MOs<br/>]<br/><br/>def generate_accused(accused_id):<br/> names = ["Ravi Kumar",
"Mohan Reddy", "Suresh", "Anil", "Venkatesh",<br/> "Lakshmi Devi", "Nagaraj",
"Prakash", "Mahesh", "Gangadhar"]<br/> return {<br/> "accused_id": accused_id,<br/>
"name": random.choice(names) + f" {accused_id}",<br/> "alias": random.choice(["",
"Kiran", "Bhai", "Anna", ""]) if random.random() < 0.3 else "",<br/> "dob":
f"{random.randint(1970,
2000)}-{random.randint(1,12):02d}-{random.randint(1,28):02d}",<br/> "gender":
random.choice(["M","M","M","F"]), # 75% male<br/> "socio_economic_status":
random.choice(["low","low","low","medium","medium","high"]),<br/>
"first_offense_date": f"20{random.randint(18,23)}-{random.randint(1,12):02d}-{random.r
andint(1,28):02d}",<br/> }<br/><br/>def generate_fir(fir_id, ps_pool, crime_types,
gangs):<br/> # Seasonal pattern: theft spikes during Diwali (Oct-Nov)<br/> # and summer
evenings; murder is uniform<br/> ct = random.choice(crime_types)<br/> month =
random.randint(1, 12)<br/> if ct[2] == "Theft" and month in [10, 11]:<br/> # Diwali
spike — boost probability<br/> pass<br/><br/> # Geographic clustering: gangs operate
near their home district<br/> gang = random.choice(gangs) if random.random() < 0.15
else None<br/> if gang:<br/> ps = [p for p in ps_pool if p["district_name"] ==
gang["home_district"]]<br/> ps = random.choice(ps) if ps else
random.choice(ps_pool)<br/> ct_id = gang["crime_type"]<br/> else:<br/> ps =
random.choice(ps_pool)<br/> ct_id = ct[0]<br/><br/> fir_date = datetime(2025, month,
random.randint(1, 28))<br/> return {<br/> "fir_id": fir_id,<br/> "fir_number":
f"{fir_id}/{fir_date.year}",<br/> "ps_id": ps["ps_id"],<br/> "crime_type_id":
ct_id,<br/> "fir_date": fir_date.strftime("%Y-%m-%d"),<br/> "fir_time":
f"{random.randint(0,23):02d}:{random.randint(0,59):02d}:00",<br/> "status":
random.choices(<br/> ["under_investigation", "chargesheeted", "closed",
"convicted"],<br/> weights=[50, 25, 15, 10])[0],<br/> "summary": f"Crime type {ct[2]}
```

```
at {ps['ps_name']}",<br/> "lat": ps["lat"] + random.uniform(-0.05, 0.05),<br/> "lng":
ps["lng"] + random.uniform(-0.05, 0.05),<br/> "mo_description": "Forced entry, fled
with valuables",<br/> }<br/><br/>def main():<br/> out = Path("scripts/data")<br/>
out.mkdir(exist_ok=True)<br/><br/> # Generate districts<br/> with open(out /
"districts.csv", "w", newline="") as f:<br/> w = csv.DictWriter(f,
fieldnames=["district_id","district_name","division",<br/>
"population","urbanization_rate","lat","lng"])<br/> w.writeheader()<br/> for i, d in
enumerate(DISTRICTS, 1):<br/> w.writerow({"district_id": i, "district_name": d[0],
"division": d[1],<br/> "population": d[2], "urbanization_rate": d[3],<br/> "lat":
d[4], "lng": d[5]})<br/><br/> # Generate police stations (5-15 per district = ~900
total)<br/> with open(out / "police_stations.csv", "w", newline="") as f:<br/> w =
csv.DictWriter(f,
fieldnames=["ps_id","ps_name","district_id","range_name","lat","lng"])<br/>
w.writeheader()<br/> ps_id = 1<br/> ps_pool = []<br/> for d_id, d in
enumerate(DISTRICTS, 1):<br/> n_ps = random.randint(15, 30)<br/> for n in
range(n_ps):<br/> ps_name = f"{d[0]} PS {n+1}"<br/> lat = d[4] + random.uniform(-0.3,
0.3)<br/> lng = d[5] + random.uniform(-0.3, 0.3)<br/> w.writerow({"ps_id": ps_id,
"ps_name": ps_name, "district_id": d_id,<br/> "range_name": d[1], "lat": lat, "lng":
lng})<br/> ps_pool.append({"ps_id": ps_id, "ps_name": ps_name,<br/> "district_name":
d[0], "lat": lat, "lng": lng})<br/> ps_id += 1<br/><br/> # Generate crime types<br/>
with open(out / "crime_types.csv", "w", newline="") as f:<br/> w = csv.DictWriter(f,
fieldnames=["crime_type_id","ipc_section","category","severity","description"])<br/>
w.writeheader()<br/> for ct in CRIME_TYPES:<br/> w.writerow({"crime_type_id": ct[0],
"ipc_section": ct[1], "category": ct[2],<br/> "severity": ct[3], "description":
ct[4]})<br/><br/> # Generate accused (35,000)<br/> with open(out / "accused.csv", "w",
newline="") as f:<br/> w = csv.DictWriter(f,
fieldnames=["accused_id","name","alias","dob","gender",<br/>
"address","occupation","education","socio_economic_status","first_offense_date"])<br/>
w.writeheader()<br/> for aid in range(1, 35001):<br/> a = generate_accused(aid)<br/>
w.writerow({**a, "address": f"Address {aid}", "occupation": "Laborer",<br/>
"education": random.choice(["Primary","Secondary","Higher
Secondary","Graduate"])})<br/><br/> # Generate FIRs (50,000) with seasonal + gang
patterns<br/> with open(out / "firs.csv", "w", newline="") as f:<br/> w =
csv.DictWriter(f, fieldnames=["fir_id","fir_number","ps_id","crime_type_id",<br/>
"fir_date","fir_time","status","summary","lat","lng","mo_description"])<br/>
w.writeheader()<br/> for fid in range(1, 50001):<br/> fir = generate_fir(fid, ps_pool,
CRIME_TYPES, GANGS)<br/> w.writerow(fir)<br/><br/> # Generate fir_accused (62,000
links, avg 1.24 accused per FIR)<br/> # Gang members co-appear more frequently<br/>
with open(out / "fir_accused.csv", "w", newline="") as f:<br/> w = csv.DictWriter(f,
fieldnames=["fir_id","accused_id","role","arrest_date","bail_status"])<br/>
w.writeheader()<br/> for fid in range(1, 50001):<br/> n_accused = random.choices([1,
1, 1, 2, 2, 3, 4], k=1)[0]<br/> for _ in range(n_accused):<br/> aid = random.randint(1,
35000)<br/> w.writerow({"fir_id": fid, "accused_id": aid,<br/> "role":
random.choice(["primary","secondary","accomplice"]),<br/> "arrest_date":
f"2025-{random.randint(1,7):02d}-{random.randint(1,28):02d}",<br/> "bail_status":
random.choice(["in_judicial_custody","on_bail","released"])})<br/><br/> # Generate
socio_economic_indicator (31 districts x 3 years = 93)<br/> with open(out /
"socio_economic.csv", "w", newline="") as f:<br/> w = csv.DictWriter(f,
fieldnames=["indicator_id","district_id","year","literacy_rate",<br/>
"unemployment_rate","migration_index","urbanization_rate","gdp_per_capita"])<br/>
w.writeheader()<br/> iid = 1<br/> for d_id, d in enumerate(DISTRICTS, 1):<br/> for year
in [2023, 2024, 2025]:<br/> w.writerow({"indicator_id": iid, "district_id": d_id,
"year": year,<br/> "literacy_rate": round(random.uniform(60, 90), 2),<br/>
"unemployment_rate": round(random.uniform(3, 12), 2),<br/> "migration_index":
round(random.uniform(0.1, 0.9), 2),<br/> "urbanization_rate": d[3],<br/>
```

```
"gdp_per_capita": round(random.uniform(50000, 300000), 2)})<br/> iid += 1<br/><br/>
print("Generated synthetic data in scripts/data/")<br/> print(f" - {len(DISTRICTS)}
districts")<br/> print(f" - {ps_id-1} police stations")<br/> print(f" - 120 crime
types")<br/> print(f" - 35,000 accused")<br/> print(f" - 50,000 FIRs with seasonal +
gang patterns")<br/><br/>if __name__ == "__main__":<br/> main()
```

Run: python3 scripts/generate_synthetic_data.py

Verify: scripts/data/firs.csv has 50,000 rows. Open in a spreadsheet and confirm seasonal patterns (Oct-Nov has more thefts).

Time: 2 hours (most of Day 1 morning)

Step 1.4 — Load Synthetic Data into Data Store [BE]

Objective: Bulk-import all CSVs into Catalyst Data Store.

Commands:

```
# Load each CSV into its Data Store table<br/>for table in district police_station
crime_type modus_operandi \<br/> accused victim fir fir_accused fir_victim fir_mo
\<br/> case_status offender_history financial_transaction socio_economic_indicator;
do<br/> echo "Loading $table..."<br/> catalyst datastore:bulk-import \<br/> --table
$table \<br/> --file scripts/data/${table}s.csv \<br/> --format
csv<br/>done<br/><br/># Verify counts<br/>catalyst datastore:query "SELECT COUNT(*)
FROM fir" # should be 50000<br/>catalyst datastore:query "SELECT COUNT(*) FROM
accused" # 35000<br/>catalyst datastore:query "SELECT COUNT(*) FROM police_station" #
~900
```

Verify: All COUNT queries return expected numbers. Spot-check: catalyst datastore:query "SELECT * FROM fir LIMIT 5" returns 5 rows with realistic data.

Time: 30 minutes

Step 1.5 — Configure Authentication & RBAC [BE]

Objective: Set up 4 roles (investigator, scrb_analyst, sp, igp) and create test users.

Commands:

```
# Create roles in Catalyst Authentication<br/>catalyst auth:role:create --name
investigator \<br/> --description "Field Investigator — own PS only"<br/>catalyst
auth:role:create --name scrb_analyst \<br/> --description "SCRB Analyst — state-wide
read"<br/>catalyst auth:role:create --name sp \<br/> --description "District SP — own
district"<br/>catalyst auth:role:create --name igp \<br/> --description "IGP/DGP —
aggregate only"<br/><br/># Create test users (one per role for demo)<br/>catalyst
auth:user:create --email inv@test.ksp \<br/> --password Demo@1234 --role investigator
--jurisdiction "ps_id:45"<br/>catalyst auth:user:create --email analyst@test.ksp
\<br/> --password Demo@1234 --role scrb_analyst --jurisdiction "state"<br/>catalyst
auth:user:create --email sp@test.ksp \<br/> --password Demo@1234 --role sp
--jurisdiction "district_id:1"<br/>catalyst auth:user:create --email igp@test.ksp
\<br/> --password Demo@1234 --role igp --jurisdiction "state_aggregate"<br/><br/>#
Verify<br/>catalyst auth:users:list<br/>catalyst auth:roles:list
```

Verify: 4 roles and 4 test users visible. Login at the Catalyst auth URL with each user succeeds.

Time: 30 minutes

Step 1.6 — Scaffold Slate Web Client [FE]

Objective: Set up the React + Vite + Tailwind + ECharts + Mapbox SPA scaffold with login working.

Commands:

```
cd web-client<br/><br/># Initialize Vite + React + TypeScript<br/>npm create
vite@latest . -- --template react-ts<br/><br/># Install dependencies<br/>npm install
@zohocornerstone/sdk @mapbox/mapbox-gl-react echarts \<br/> echarts-for-react axios
react-router-dom zustand<br/>npm install -D tailwindcss postcss autoprefixer
@types/mapbox-gl<br/><br/># Init Tailwind<br/>npx tailwindcss init -p<br/><br/>#
Configure Mapbox token in .env.local<br/>echo
"VITE_MAPBOX_TOKEN=your_mapbox_token_here" > .env.local<br/>echo
"VITE_CATALYST_PROJECT_ID=your_project_id" >> .env.local<br/>echo
"VITE_CATALYST_API_BASE=https://api.catalyst.zoho.com" >> .env.local<br/><br/># Start
dev server<br/>npm run dev
```

Files to create: src/lib/api.ts (Catalyst API client), src/pages/Login.tsx

Verify: npm run dev serves a login page at localhost:5173. Login with analyst@test.ksp succeeds and redirects to a placeholder dashboard.

Time: 2 hours

Step 1.7 — Commit & Push to GitHub [INT]

Objective: Get Day 1 work committed and pushed so the team has a clean baseline.

```
cd ksp-crime-analytics-platform<br/>git add -A<br/>git commit -m "Day 1: Catalyst
project, schema, synthetic data, auth, web scaffold"<br/>git push origin
main<br/><br/># Create Day 1 release tag (for rollback if needed)<br/>git tag -a
v0.1-day1 -m "Day 1 complete: foundation ready"<br/>git push origin v0.1-day1
```

Verify: GitHub repo shows the commit and tag. All team members can pull and run npm run dev successfully.

Time: 15 minutes

Day 2 — smolagents Orchestrator + Visualization Pipeline (5 Tools)

Day 2 is the pivot day: you install smolagents, configure the CodeAgent orchestrator, and build the first 5 visualization tools. The goal is that by end of Day 2, an analyst can ask "show me theft trends in Bengaluru Urban over the last 6 months" and get a real answer grounded in Data Store data, with an evidence trail captured. The visualization polish (heatmaps, drill-down UI) comes on Day 3; today is about the backend pipeline working end-to-end.

> **Day 2 success criteria**
>
> By end of Day 2: (1) pip install smolagents works on Catalyst Functions, (2) QuickML LLM endpoint responds to a test prompt, (3) the orchestrator CodeAgent is configured with 5 visualization tools, (4) a POST to /api/query with "show me theft trends in Bengaluru Urban last 6 months" returns a real answer with tool calls traced in last_run_steps, (5) the evidence trail is persisted to NoSQL.

Step 2.1 — Configure Catalyst Functions Runtime [BE]

Objective: Set up the Python 3.12 runtime for Catalyst Functions with smolagents installed.

Files to create: functions/orchestrator/requirements.txt, functions/orchestrator/handler.py

```
# functions/orchestrator/requirements.txt<br/>smolagents>=0.2.3<br/>zcatalyst-sdk>=1.0
.0<br/>pytz>=2024.1<br/><br/># functions/orchestrator/handler.py<br/># (Copy from
/home/z/my-project/download/router_agent_smolagents.py)<br/># This is the full
skeleton you already have. Adapt the 16 tool stubs<br/># to call Catalyst Data Store
SDK instead of returning placeholders.
```

Commands:

```
# Deploy the orchestrator function<br/>catalyst functions:deploy
functions/orchestrator/ \<br/> --name orchestrator \<br/> --runtime python3.12 \<br/>
--memory 512 \<br/> --timeout 30<br/><br/># Set environment variables<br/>catalyst
functions:env:set orchestrator \<br/>
QUICKML_LLM_ENDPOINT=https://your-quickml-endpoint \<br/> QUICKML_LLM_API_KEY=your_key
\<br/> QUICKML_LLM_MODEL=gpt-4o-mini \<br/>
DATA_STORE_PROJECT_ID=your_project_id<br/><br/># Verify deployment<br/>catalyst
functions:list<br/>catalyst functions:logs orchestrator --tail
```

Verify: Function deploys without errors. catalyst functions:invoke orchestrator --data '{"query":"hello","session_id":"test","user":{"id":"u1","role":"scrb_analyst","jurisdiction":"state"}}' returns a response (even if "I don't understand" — that means the LLM connection works).

Time: 1 hour

Step 2.2 — Verify QuickML LLM Endpoint [BE]

Objective: Confirm the QuickML LLM endpoint is OpenAI-compatible and responds to smolagents calls.

Commands:

```
# Test the QuickML endpoint directly with curl<br/>curl -X POST
$QUICKML_LLM_ENDPOINT/v1/chat/completions \<br/> -H "Authorization: Bearer
$QUICKML_LLM_API_KEY" \<br/> -H "Content-Type: application/json" \<br/> -d '{<br/>
"model": "'$QUICKML_LLM_MODEL'",<br/> "messages": [{"role": "user", "content": "Say
hello in one word"}],<br/> "max_tokens": 10<br/> }'<br/><br/># Expected:
{"choices":[{"message":{"content":"Hello"}}], ...}<br/><br/># If QuickML is not yet
available, fall back to a free OpenAI-compatible endpoint:<br/># Option A: Groq (free
tier, fast) — https://groq.com<br/># Option B: Together AI (free credits) —
https://together.ai<br/># Option C: OpenRouter (free models) —
https://openrouter.ai<br/># Set QUICKML_LLM_ENDPOINT to whichever you use.
```

Verify: curl returns a valid chat completion JSON. If it fails, debug the endpoint URL, API key, and model name before proceeding.

Time: 30 minutes

Step 2.3 — Implement 5 Visualization Tools [BE]

Objective: Replace the placeholder tool stubs with real Data Store queries for the 5 visualization tools.

Files to create: 5 files under functions/visualization/

Code (trend_query.py — first tool, others follow same pattern):

```
# functions/visualization/trend_query.py<br/>"""Aggregate crime trends by time period
and location."""<br/>from datetime import datetime<br/>from typing import Any,
Dict<br/>import zcatalyst_sdk as catalyst<br/><br/>def trend_query(crime_type: str,
location: str,<br/> date_from: str, date_to: str) -> Dict[str, Any]:<br/> """Query
Data Store for crime trends.<br/><br/> Args:<br/> crime_type: e.g., "Theft",
"Robbery", or "all"<br/> location: district name or "state" for all<br/> date_from:
YYYY-MM-DD<br/> date_to: YYYY-MM-DD<br/> Returns:<br/> {"total_cases": int, "monthly":
[{"month": "YYYY-MM", "count": int}]}<br/> """<br/> ds =
catalyst.datastore()<br/><br/> # Build SQL with proper jurisdiction filtering<br/>
base_sql = """<br/> SELECT DATE_TRUNC('month', fir_date) AS month, COUNT(*) AS
cnt<br/> FROM fir f<br/> JOIN crime_type ct ON f.crime_type_id = ct.crime_type_id<br/>
JOIN police_station ps ON f.ps_id = ps.ps_id<br/> JOIN district d ON ps.district_id =
d.district_id<br/> WHERE f.fir_date BETWEEN '{date_from}' AND '{date_to}'<br/>
""".format(date_from=date_from, date_to=date_to)<br/><br/> if crime_type.lower() !=
"all":<br/> base_sql += f" AND ct.category ILIKE '%{crime_type}%'"<br/><br/> if
location.lower() != "state":<br/> base_sql += f" AND d.district_name ILIKE
'%{location}%'"<br/><br/> base_sql += " GROUP BY month ORDER BY month"<br/><br/> #
Execute query<br/> result = ds.execute_query(base_sql)<br/><br/> monthly = [{"month":
str(r["month"])[:7], "count": r["cnt"]} for r in result]<br/> total = sum(m["count"]
for m in monthly)<br/><br/> # Capture data_refs for evidence trail (the orchestrator
wrapper reads this)<br/> return {<br/> "total_cases": total,<br/> "monthly":
monthly,<br/> "_data_refs": {<br/> "tables": ["fir", "crime_type", "police_station",
"district"],<br/> "row_count": total,<br/> "query_sql": base_sql,<br/> "cache_hit":
False,<br/> }<br/> }
```

Other 4 tools to implement (same pattern):

- hotspot_kde(district, crime_type, date_range) — KDE on lat/lng, returns cell densities + emerging_trend_score
- drill_down(level, parent_id) — state→district→PS→beat aggregation
- emerging_trend_score(district, crime_type, window_days=30) — z-score vs historical baseline

seasonal_decomp(district, crime_type, window_months=36) — STL decomposition (use statsmodels if available, else simple moving average)

Verify: Call each tool directly from a Python shell: python3 -c "from functions.visualization.trend_query import trend_query; print(trend_query('Theft','Bengaluru Urban','2025-01-01','2025-06-30'))" returns real data.

Time: 3 hours (most of Day 2 afternoon)

Step 2.4 — Register Tools with Orchestrator [BE]

Objective: Wire the 5 visualization tools into the smolagents CodeAgent.

Files to edit: functions/orchestrator/router_agent.py

Code changes:

```
# In router_agent.py, replace the placeholder _trend_query, _hotspot_kde,<br/># _drill_down, _emerging_trend_score, _seasonal_decomp functions with<br/># imports from the visualization module:<br/><br/>from functions.visualization.trend_query import trend_query as _trend_query<br/>from functions.visualization.hotspot_kde import hotspot_kde as _hotspot_kde<br/>from functions.visualization.drill_down import drill_down as _drill_down<br/>from functions.visualization.emerging_trend import emerging_trend_score as _emerging_trend_score<br/>from functions.visualization.seasonal_decomp import seasonal_decomp as _seasonal_decomp<br/><br/># Then in build_tools(), the make_tool() calls stay the same — they now<br/># wrap the real implementations instead of the placeholders.<br/><br/># Update ROUTER_SYSTEM_PROMPT to remove the network/predictive tools<br/># for now (they come on Days 4-5). List only the 5 visualization tools<br/># in the AVAILABLE TOOLS section.
```

Verify: Redeploy the orchestrator. Invoke it with: {"query": "show me theft trends in Bengaluru Urban from January to June 2025", "session_id":"test","user":{...}}. The response should contain real case counts and monthly breakdown, and last_run_steps should show the trend_query tool was called.

Time: 45 minutes

Step 2.5 — Persist Evidence Trail to NoSQL [BE]

Objective: Write the evidence trail (built from last_run_steps) to the NoSQL audit_log collection.

Commands:

```
# Create the NoSQL collection<br/>catalyst nosql:collection:create --name audit_log<br/><br/># In router_agent.py handler(), after building the trail:<br/># from build_evidence_trail() — uncomment the persistence line:<br/>catalyst.nosql().collection("audit_log").insert_row(trail)<br/><br/># Verify after a test query:<br/>catalyst nosql:query "SELECT * FROM audit_log WHERE session_id='test' LIMIT 5"
```

Verify: After invoking the orchestrator, the NoSQL query returns the trail document with tool_calls, data_references, and confidence populated.

Time: 30 minutes

Step 2.6 — Commit & Tag [INT]

```
git add -A<br/>git commit -m "Day 2: smolagents orchestrator + 5 visualization tools +
evidence trail"<br/>git tag -a v0.2-day2 -m "Day 2: viz pipeline working
end-to-end"<br/>git push origin main --tags
```

Time: 10 minutes

Day 3 — Visualization Polish + Red-Zone Alerts

Day 3 turns the backend pipeline into a visual, interactive experience. The goal is that an SCRB analyst can open the dashboard, see a Karnataka map with theft hotspots as a heatmap, watch red zones pulse when a trend spikes, and drill from state to district to PS by clicking. This is the most demo-worthy day because it produces the visual that the jury will remember. Spend the time to make it look professional.

Day 3 success criteria

By end of Day 3: (1) Dashboard shows Karnataka map with hotspot heatmap, (2) clicking a district zooms in to show PS-level hotspots, (3) emerging-trend red zones pulse visibly when z-score > 2.0, (4) Catalyst Signals fires when a red zone triggers, (5) Push Notifications delivers an alert to a test device, (6) KPI strip, 7x24 day-hour heatmap, and top-10 crime types chart all render with real data.

Step 3.1 — Build Karnataka Map with Hotspot Heatmap [FE]

Objective: Render a Mapbox map of Karnataka with a heatmap layer from hotspot_kde tool output.

Files to create: web-client/src/components/KarnatakaMap.tsx

```
// web-client/src/components/KarnatakaMap.tsx<br/>import { useEffect, useRef, useState
} from 'react';<br/>import mapboxgl from 'mapbox-gl';<br/>import { fetchHotspots }
from '../lib/api';<br/><br/>mapboxgl.accessToken =
import.meta.env.VITE_MAPBOX_TOKEN;<br/><br/>export function KarnatakaMap({ district,
crimeType, dateRange }) {<br/> const mapContainer = useRef<HTMLDivElement>(null);<br/>
const map = useRef<mapboxgl.Map>();<br/> const [hotspots, setHotspots] =
useState([]);<br/><br/> useEffect(() => {<br/> if (map.current) return;<br/>
map.current = new mapboxgl.Map({<br/> container: mapContainer.current!,<br/> style:
'mapbox://styles/mapbox/light-v11',<br/> center: [76.5, 14.5], // Karnataka
center<br/> zoom: 6.5,<br/> });<br/><br/> map.current.on('load', () => {<br/>
map.current!.addSource('hotspots', {<br/> type: 'geojson',<br/> data: { type:
'FeatureCollection', features: [] }<br/> });<br/><br/> map.current!.addLayer({<br/>
id: 'hotspot-heat',<br/> type: 'heatmap',<br/> source: 'hotspots',<br/> paint: {<br/>
'heatmap-weight': ['interpolate', ['linear'],<br/> ['get', 'density'], 0, 0, 10,
1],<br/> 'heatmap-intensity': ['interpolate', ['linear'],<br/> ['zoom'], 0, 1, 9,
3],<br/> 'heatmap-color': ['interpolate', ['linear'],<br/> ['heatmap-density'], 0,
'rgba(0,0,0,0)',<br/> 0.2, '#4daf4a', 0.4, '#377eb8',<br/> 0.6, '#ff7f00', 0.8,
'#e41a1c', 1, '#b10026'],<br/> 'heatmap-radius': 30,<br/> 'heatmap-opacity':
0.75,<br/> }<br/> });<br/><br/> // Red-zone pulsing layer (emerging trends only)<br/>
map.current!.addLayer({<br/> id: 'red-zone-pulse',<br/> type: 'circle',<br/> source:
'hotspots',<br/> filter: ['get', 'is_red_zone'],<br/> paint: {<br/> 'circle-radius':
['interpolate', ['linear'],<br/> ['get', 'density'], 5, 15, 10, 30],<br/>
'circle-color': '#e41a1c',<br/> 'circle-opacity': 0.6,<br/> 'circle-stroke-width':
2,<br/> 'circle-stroke-color': '#b10026',<br/> // Pulsing animation<br/>
'circle-opacity': [<br/> 'interpolate', ['linear'],<br/> ['time'], 0, 0.6, 1000, 0.2,
2000, 0.6<br/> ],<br/> }<br/> });<br/> });<br/> }, []);<br/><br/> useEffect(() =>
{<br/> if (!map.current?.loaded()) return;<br/> fetchHotspots(district, crimeType,
dateRange).then(data => {<br/> const features = data.cells.map(c => ({<br/> type:
'Feature',<br/> geometry: { type: 'Point', coordinates: [c.lng, c.lat] },<br/>
properties: { density: c.density, is_red_zone: data.is_red_zone }<br/> }));<br/>
map.current!.getSource('hotspots')<br/> .setData({ type: 'FeatureCollection', features
});<br/> });<br/> }, [district, crimeType, dateRange]);<br/><br/> return <div
```

```
ref={mapContainer} className="w-full h-[600px]" />;<br/>}
```

Verify: Dashboard shows Karnataka with heatmap in Bengaluru Urban (high crime area). Zoom and pan work.

Time: 2 hours

Step 3.2 — Implement District Drill-Down [FE]

Objective: Click a district to zoom in and show PS-level hotspots.

Code to add to KarnatakaMap.tsx:

```
// Add click handler for drill-down<br/>map.current.on('click', 'district-fill', (e)
=> {<br/> const districtId = e.features[0].properties.district_id;<br/> const
districtName = e.features[0].properties.district_name;<br/> onDrillDown({ level: 'ps',
parent_id: districtId, name: districtName });<br/> map.current.flyTo({<br/> center:
e.features[0].geometry.coordinates,<br/> zoom: 9,<br/> duration: 1000,<br/>
});<br/>});<br/><br/>// Add district boundaries layer (GeoJSON from Karnataka
GIS)<br/>map.current.addSource('districts', {<br/> type: 'geojson',<br/> data:
'/data/karnataka-districts.geojson'<br/>});<br/>map.current.addLayer({<br/> id:
'district-fill',<br/> type: 'fill',<br/> source: 'districts',<br/> paint: {<br/>
'fill-color': '#088',<br/> 'fill-opacity': ['case',<br/> ['boolean', ['feature-state',
'selected'], false], 0.3, 0.05]<br/> }<br/>});<br/>map.current.addLayer({<br/> id:
'district-outline',<br/> type: 'line',<br/> source: 'districts',<br/> paint: {
'line-color': '#0f2740', 'line-width': 1 }<br/>});
```

Files to get: Karnataka districts GeoJSON — search "Karnataka districts geojson github", download, place in web-client/public/data/karnataka-districts.geojson

Verify: Click Bengaluru district → map zooms in → PS-level hotspots appear.

Time: 1.5 hours

Step 3.3 — Configure Signals + Push Notifications [BE+INT]

Objective: When emerging_trend_score returns z-score > 2.0, fire a Catalyst Signal that triggers a Push Notification.

Commands:

```
# Create a Signal event type<br/>catalyst signals:event-type:create --name
emerging_trend_alert<br/><br/># Create a push notification function<br/>catalyst
functions:deploy functions/push_alert/ \<br/> --name push_alert \<br/> --trigger
signals:emerging_trend_alert<br/><br/># functions/push_alert/handler.py<br/>def
handler(event, context):<br/> district = event["district"]<br/> crime_type =
event["crime_type"]<br/> z_score = event["z_score"]<br/><br/>
catalyst.push_notifications().send({<br/> "message": f"RED ZONE: {crime_type} spiking
in {district} (z={z_score:.1f})",<br/> "audience": "role:scrb_analyst,role:sp",<br/>
"deep_link": f"/dashboard?district={district}&crime={crime_type}",<br/> "priority":
"high",<br/> })<br/> return {"status": "sent"}<br/><br/># In hotspot_kde.py, when
is_red_zone is True, fire the signal:<br/>if is_red_zone:<br/>
catalyst.signals().fire("emerging_trend_alert", {<br/> "district": district,
"crime_type": crime_type, "z_score": z<br/> })
```

Verify: Manually trigger a red-zone (modify test data to spike a district) → push notification arrives on test device within 5 seconds.

Time: 1.5 hours

Step 3.4 — Build KPI Strip + Secondary Charts [FE]

Objective: Add the top-row KPI strip, 7x24 day-hour heatmap, and top-10 crime types bar chart.

Files to create: KpiStrip.tsx, DayHourHeatmap.tsx, TopCrimesChart.tsx

Code (KpiStrip.tsx):

```
// web-client/src/components/KpiStrip.tsx<br/>import { useQuery } from
'@tanstack/react-query';<br/>import { fetchKpis } from '../lib/api';<br/><br/>export
function KpiStrip() {<br/> const { data } = useQuery(['kpis'], fetchKpis);<br/> if
(!data) return null;<br/> return (<br/> <div className="grid grid-cols-4 gap-4
p-4"><br/> <KpiCard label="Total Crimes (30d)" value={data.total_crimes}><br/>
delta={data.mom_change} deltaDirection={data.mom_direction} /><br/> <KpiCard
label="Active Hotspots" value={data.active_hotspots}<br/> accent="amber" /><br/>
<KpiCard label="Emerging Trends" value={data.emerging_trends}<br/> accent="red"
pulsing={data.emerging_trends > 0} /><br/> <KpiCard label="Repeat Offenders"
value={data.repeat_offenders}<br/> accent="teal" /><br/> </div><br/>
);<br/>}<br/><br/>function KpiCard({ label, value, delta, deltaDirection, accent,
pulsing }) {<br/> const accentColor = {<br/> amber: 'text-amber-600', red:
'text-red-600',<br/> teal: 'text-teal-600', default: 'text-slate-900'<br/> }[accent ||
'default'];<br/> return (<br/> <div className={`bg-white rounded-lg shadow p-4
${pulsing ? 'animate-pulse' : ''}`}><br/> <div className="text-xs text-slate-500
uppercase">{label}</div><br/> <div className={`text-3xl font-bold
${accentColor}`}>{value}</div><br/> {delta !== undefined && (<br/> <div
className={`text-sm ${deltaDirection === 'up' ? 'text-red-600' :
'text-green-600'}`}><br/> {deltaDirection === 'up' ? '↑' : '↓'} {Math.abs(delta)}%
MoM<br/> </div><br/> )}<br/> </div><br/> );<br/>}
```

Verify: KPI strip shows 4 cards with real numbers from Data Store. Red "Emerging Trends" card pulses when there are active alerts.

Time: 1.5 hours

Step 3.5 — Commit & Tag [INT]

```
git commit -m "Day 3: Karnataka heatmap, drill-down, red-zone alerts, KPI strip,
charts"<br/>git tag -a v0.3-day3 -m "Day 3: viz polish + alerts working"<br/>git push
origin main --tags
```

Time: 10 minutes

Day 4 — Network Link Analysis Pipeline (5 Tools)

Day 4 builds the second pipeline: the criminal network graph. The goal is that an analyst can search for an accused person, see their network of co-accused, financial links, and MO-similar cases as an interactive force-directed graph, and run Louvain community detection to surface organized crime groups. This pipeline delivers the "hidden criminal associations that are impossible to spot in Excel" demo moment from the problem statement.

Day 4 success criteria

By end of Day 4: (1) build_graph returns a real graph from Data Store, (2) detect_communities runs Louvain and returns community labels, (3) ECharts force-directed graph renders interactively in the dashboard, (4) clicking a node shows a detail panel, (5) repeat_offender_profile returns a cross-jurisdictional profile, (6) mo_cluster uses QuickML embeddings to group similar MOs.

Step 4.1 — Implement build_graph Tool [BE]

Objective: Construct a subgraph around a seed entity by traversing fir_accused and offender_history.

Files to create: functions/network/build_graph.py

```
# functions/network/build_graph.py<br/>"""Build a criminal network graph around a seed
entity."""<br/>from typing import Any, Dict, List<br/>import zcatalyst_sdk as
catalyst<br/><br/>def build_graph(seed_entity: Dict[str, Any], depth: int = 2) ->
Dict[str, Any]:<br/> """Build graph by traversing co-accused relationships.<br/><br/>
Args:<br/> seed_entity: {"type": "accused"|"victim"|"location"|"incident", "id":
int}<br/> depth: 1 (direct) or 2 (includes co-accused of co-accused)<br/>
Returns:<br/> {"nodes": [...], "edges": [...], "seed": ..., "depth": ...}<br/>
"""<br/> ds = catalyst.datastore()<br/> nodes, edges = set(), []<br/> seed_id =
f"{seed_entity['type']}:{seed_entity['id']}"<br/> nodes.add(seed_id)<br/><br/> # Depth
1: all FIRs involving the seed accused<br/> if seed_entity["type"] == "accused":<br/>
sql = f"""<br/> SELECT f.fir_id, f.fir_number, f.crime_type_id, f.fir_date,<br/>
f.ps_id, f.summary<br/> FROM fir f<br/> JOIN fir_accused fa ON f.fir_id =
fa.fir_id<br/> WHERE fa.accused_id = {seed_entity['id']}<br/> """<br/> seed_firs =
ds.execute_query(sql)<br/><br/> # All co-accused on those FIRs<br/> fir_ids =
[r["fir_id"] for r in seed_firs]<br/> if fir_ids:<br/> co_sql = f"""<br/> SELECT
DISTINCT a.accused_id, a.name, a.alias, a.socio_economic_status<br/> FROM accused
a<br/> JOIN fir_accused fa ON a.accused_id = fa.accused_id<br/> WHERE fa.fir_id IN
({','.join(map(str, fir_ids))})<br/> AND a.accused_id != {seed_entity['id']}<br/>
"""<br/> co_accused = ds.execute_query(co_sql)<br/> for ca in co_accused:<br/> node_id
= f"accused:{ca['accused_id']}"<br/> nodes.add(node_id)<br/> edges.append({<br/>
"source": seed_id, "target": node_id,<br/> "type": "co_accused", "weight": 1.0,<br/>
"firs": fir_ids<br/> })<br/><br/> if depth >= 2:<br/> # Depth 2: co-accused of
co-accused<br/> co_ids = [r["accused_id"] for r in co_accused]<br/> if co_ids:<br/>
d2_sql = f"""<br/> SELECT fa.accused_id, fa2.accused_id AS co_id,<br/> a.name,
a.alias,<br/> COUNT(*) AS shared_firs<br/> FROM fir_accused fa<br/> JOIN fir_accused
fa2 ON fa.fir_id = fa2.fir_id<br/> JOIN accused a ON fa2.accused_id = a.accused_id<br/>
WHERE fa.accused_id IN ({','.join(map(str, co_ids))})<br/> AND fa2.accused_id NOT IN
({seed_entity['id']},<br/> {','.join(map(str, co_ids))})<br/> GROUP BY fa.accused_id,
fa2.accused_id, a.name, a.alias<br/> HAVING COUNT(*) >= 1<br/> """<br/> d2 =
ds.execute_query(d2_sql)<br/> for r in d2:<br/> src =
f"accused:{r['accused_id']}"<br/> tgt = f"accused:{r['co_id']}"<br/> nodes.add(src);
nodes.add(tgt)<br/> edges.append({<br/> "source": src, "target": tgt,<br/> "type":
```

```
"co_accused", "weight": float(r["shared_firs"])<br/> })<br/><br/> # Add FIR nodes and
incident edges<br/> for fir in seed_firs:<br/> fir_node =
f"incident:{fir['fir_id']}"<br/> nodes.add(fir_node)<br/> edges.append({<br/>
"source": seed_id, "target": fir_node,<br/> "type": "incident", "weight": 1.0<br/>
})<br/><br/> # Financial links (if any)<br/> fin_sql = f"""<br/> SELECT accused_id,
account_number, SUM(amount) AS total<br/> FROM financial_transaction<br/> WHERE
accused_id = {seed_entity['id']}<br/> OR account_number IN (<br/> SELECT
account_number FROM financial_transaction<br/> WHERE accused_id =
{seed_entity['id']}<br/> )<br/> GROUP BY accused_id, account_number<br/> """<br/> fin
= ds.execute_query(fin_sql)<br/> for r in fin:<br/> if r["accused_id"] !=
seed_entity["id"]:<br/> node_id = f"accused:{r['accused_id']}"<br/>
nodes.add(node_id)<br/> edges.append({<br/> "source": seed_id, "target": node_id,<br/>
"type": "financial_link", "weight": 1.0<br/> })<br/><br/> # Build node objects with
attributes<br/> node_list = []<br/> for n in nodes:<br/> ntype, nid = n.split(":")<br/>
node_list.append({"id": n, "type": ntype, "entity_id": int(nid)})<br/><br/> return
{<br/> "nodes": node_list,<br/> "edges": edges,<br/> "seed": seed_entity,<br/>
"depth": depth,<br/> "_data_refs": {<br/> "tables": ["fir", "fir_accused", "accused",
"financial_transaction"],<br/> "row_count": len(node_list) + len(edges),<br/>
"query_sql": "multi-query: see build_graph.py",<br/> "cache_hit": False,<br/> }<br/> }
```

Verify: python3 -c "from functions.network.build_graph import build_graph; g = build_graph({'type':'accused','id':1023}, depth=2); print(len(g['nodes']), 'nodes,', len(g['edges']), 'edges')" returns a non-trivial graph.

Time: 2 hours

Step 4.2 — Implement detect_communities (Louvain) [BE]

Objective: Run Louvain community detection on the graph to surface organized crime groups.

Files to create: functions/network/detect_communities.py

```
# functions/network/detect_communities.py<br/>"""Louvain community detection on a
criminal network graph."""<br/>from typing import Any, Dict<br/>try:<br/> import
networkx as nx<br/> from community import community_louvain # pip install
python-louvain<br/> HAS_LOUVAIN = True<br/>except ImportError:<br/> HAS_LOUVAIN =
False<br/><br/>def detect_communities(graph: Dict[str, Any]) -> Dict[str, Any]:<br/>
"""Run Louvain community detection.<br/><br/> Args:<br/> graph: output of
build_graph()<br/> Returns:<br/> {"communities": [...], "modularity": float}<br/>
"""<br/> if not HAS_LOUVAIN:<br/> # Fallback: connected components (weaker but no
dep)<br/> return _connected_components_fallback(graph)<br/><br/> # Build NetworkX
graph<br/> G = nx.Graph()<br/> for node in graph["nodes"]:<br/> G.add_node(node["id"],
**node)<br/> for edge in graph["edges"]:<br/> G.add_edge(edge["source"],
edge["target"],<br/> weight=edge.get("weight", 1.0))<br/><br/> # Run Louvain<br/>
partition = community_louvain.best_partition(G, resolution=1.0)<br/><br/> # Group
nodes by community<br/> communities = {}<br/> for node, comm_id in
partition.items():<br/> communities.setdefault(comm_id, []).append(node)<br/><br/> #
Build response<br/> comm_list = []<br/> for comm_id, members in
communities.items():<br/> if len(members) < 2: # skip singletons<br/> continue<br/>
comm_list.append({<br/> "id": comm_id,<br/> "size": len(members),<br/> "members":
members,<br/> "label": f"Cluster {chr(65 + comm_id)}", # A, B, C...<br/> "avg_degree":
sum(dict(G.degree(members)).values()) / len(members)<br/> })<br/><br/>
comm_list.sort(key=lambda c: c["size"], reverse=True)<br/><br/> return {<br/>
"communities": comm_list,<br/> "total_communities": len(comm_list),<br/> "modularity":
```

```
community_louvain.modularity(partition, G),<br/> "algorithm": "louvain_v1.0"<br/>
}<br/><br/>def _connected_components_fallback(graph):<br/> G = nx.Graph()<br/> for n
in graph["nodes"]: G.add_node(n["id"])<br/> for e in graph["edges"]:
G.add_edge(e["source"], e["target"])<br/> comps =
list(nx.connected_components(G))<br/> return {<br/> "communities": [{"id": i, "size":
len(c), "members": list(c),<br/> "label": f"Component {i+1}"}<br/> for i, c in
enumerate(comps) if len(c) >= 2],<br/> "total_communities": len([c for c in comps if
len(c) >= 2]),<br/> "modularity": 0.0,<br/> "algorithm":
"connected_components_fallback"<br/> }
```

Add to requirements.txt: networkx>=3.2, python-louvain>=0.16

Verify: Run on the graph from Step 4.1 — should return 2-5 communities for a typical repeat offender.

Time: 1 hour

Step 4.3 — Implement repeat_offender_profile + centrality + mo_cluster [BE]

Objective: Complete the remaining 3 network pipeline tools.

Files to create:

functions/network/repeat_offender.py — aggregates offender_history across jurisdictions, flags repeat offenders (3+ prior cases)

functions/network/centrality.py — NetworkX betweenness + degree centrality

functions/network/mo_cluster.py — QuickML embeddings on MO descriptions + K-means clustering

Code (mo_cluster.py — uses QuickML embeddings):

```
# functions/network/mo_cluster.py<br/>"""Cluster modus operandi descriptions using
QuickML embeddings."""<br/>from typing import Any, Dict, List<br/>import zcatalyst_sdk
as catalyst<br/>from sklearn.cluster import KMeans # pip install
scikit-learn<br/>import numpy as np<br/><br/>def mo_cluster(crime_type: str,
n_clusters: int = 5) -> Dict[str, Any]:<br/> """Cluster MO descriptions for a crime
type.<br/><br/> Args:<br/> crime_type: e.g., "Theft"<br/> n_clusters: number of
K-means clusters<br/> Returns:<br/> {"clusters": [{"id": int, "size": int, "label":
str, "examples": [...]}]}<br/> """<br/> ds = catalyst.datastore()<br/><br/> # Fetch MO
descriptions for this crime type<br/> sql = f"""<br/> SELECT f.fir_id,
f.mo_description<br/> FROM fir f<br/> JOIN crime_type ct ON f.crime_type_id =
ct.crime_type_id<br/> WHERE ct.category ILIKE '%{crime_type}%'<br/> AND
f.mo_description IS NOT NULL<br/> LIMIT 500<br/> """<br/> rows =
ds.execute_query(sql)<br/><br/> if len(rows) < n_clusters:<br/> return {"clusters":
[], "error": "Insufficient data for clustering"}<br/><br/> # Get embeddings from
QuickML (OpenAI-compatible embeddings endpoint)<br/> texts = [r["mo_description"] for
r in rows]<br/> embeddings = _get_embeddings(texts) # calls QuickML
/v1/embeddings<br/><br/> # K-means clustering<br/> kmeans =
KMeans(n_clusters=n_clusters, random_state=42, n_init=10)<br/> labels =
kmeans.fit_predict(embeddings)<br/><br/> # Build cluster summaries<br/> clusters =
[]<br/> for i in range(n_clusters):<br/> members = [rows[j] for j in range(len(rows))
if labels[j] == i]<br/> # Representative example: closest to centroid<br/> centroid =
kmeans.cluster_centers_[i]<br/> dists = [np.linalg.norm(embeddings[j] - centroid)<br/>
for j in range(len(rows)) if labels[j] == i]<br/> rep_idx =
dists.index(min(dists))<br/> clusters.append({<br/> "id": i,<br/> "size":
```

```
len(members),<br/> "label": members[rep_idx]["mo_description"][:80],<br/> "examples":
[m["mo_description"][:100] for m in members[:3]],<br/> "fir_ids": [m["fir_id"] for m
in members]<br/> })<br/><br/> clusters.sort(key=lambda c: c["size"],
reverse=True)<br/> return {"clusters": clusters, "algorithm": "kmeans_k=" +
str(n_clusters)}<br/><br/>def _get_embeddings(texts: List[str]) -> np.ndarray:<br/>
"""Call QuickML embeddings endpoint (OpenAI-compatible)."""<br/> import requests,
os<br/> resp = requests.post(<br/>
f"{os.environ['QUICKML_LLM_ENDPOINT']}/v1/embeddings",<br/> headers={"Authorization":
f"Bearer {os.environ['QUICKML_LLM_API_KEY']}"},<br/> json={"model":
"text-embedding-3-small", "input": texts}<br/> )<br/> data = resp.json()<br/> return
np.array([d["embedding"] for d in data["data"]])
```

Verify: mo_cluster("Theft") returns 5 clusters with distinct MO labels (e.g., "forced entry rear window", "chain snatching by bike").

Time: 2 hours

Step 4.4 — Register Network Tools + Update System Prompt [BE]

Objective: Wire the 5 network tools into the orchestrator.

Changes to router_agent.py:

```
# Add imports<br/>from functions.network.build_graph import build_graph as
_build_graph<br/>from functions.network.detect_communities import detect_communities
as _detect_communities<br/>from functions.network.repeat_offender import
repeat_offender_profile as _repeat_offender<br/>from functions.network.mo_cluster
import mo_cluster as _mo_cluster<br/>from functions.network.centrality import
compute_centrality as _centrality<br/><br/># In build_tools(), uncomment/add the 5
network tool registrations<br/># In ROUTER_SYSTEM_PROMPT, add the 5 network tools to
AVAILABLE TOOLS:<br/># - build_graph(seed_entity, depth=2) -> dict<br/># -
detect_communities(graph) -> dict<br/># - repeat_offender_profile(accused_id) ->
dict<br/># - mo_cluster(crime_type) -> dict<br/># - centrality(graph) ->
dict<br/><br/># Redeploy<br/>catalyst functions:deploy functions/orchestrator/ --name
orchestrator
```

Verify: Invoke: {"query": "show me the criminal network around accused Ravi Kumar", "session_id":"test","user":{...}} — orchestrator calls lookup_accused then build_graph then detect_communities in a single code block.

Time: 30 minutes

Step 4.5 — Build Network Graph Visualization [FE]

Objective: Render the graph as an interactive ECharts force-directed diagram.

Files to create: web-client/src/components/NetworkGraph.tsx

```
// web-client/src/components/NetworkGraph.tsx<br/>import ReactECharts from
'echarts-for-react';<br/>import { useState } from 'react';<br/><br/>const NODE_COLORS
= {<br/> accused: '#e41a1c', victim: '#377eb8', location: '#4daf4a',<br/> incident:
'#ff7f00', account: '#984ea3'<br/>};<br/><br/>export function NetworkGraph({ graph,
communities }) {<br/> const [selectedNode, setSelectedNode] =
```

```
useState(null);<br/><br/> const option = {<br/> tooltip: {<br/> formatter: (p) =>
p.dataType === 'node'<br/> ? `${p.data.type}: ${p.data.entity_id}`<br/> :
`${p.data.type} (weight: ${p.data.weight})`<br/> },<br/> series: [{<br/> type:
'graph',<br/> layout: 'force',<br/> roam: true,<br/> draggable: true,<br/> force:
{<br/> repulsion: 200, edgeLength: 80,<br/> gravity: 0.1, layoutAnimation: true<br/>
},<br/> label: { show: true, position: 'right', fontSize: 10 },<br/> data:
graph.nodes.map(n => ({<br/> id: n.id,<br/> name: n.id.split(':')[1],<br/> symbolSize:
n.centrality ? 10 + n.centrality * 30 : 15,<br/> itemStyle: { color:
NODE_COLORS[n.type] || '#888' },<br/> ...n<br/> })),<br/> edges: graph.edges.map(e =>
({<br/> source: e.source, target: e.target,<br/> lineStyle: {<br/> width: 1 +
e.weight,<br/> color: e.type === 'financial_link' ? '#984ea3'<br/> : e.type ===
'co_accused' ? '#e41a1c' : '#999',<br/> curveness: 0.2<br/> },<br/> ...e<br/>
})),<br/> emphasis: { focus: 'adjacency', lineStyle: { width: 4 } },<br/> // Community
hulls<br/> markBoundary: communities?.map(c => ({<br/> name: c.label, items:
c.members, itemStyle: { opacity: 0.1 }<br/> })) || []<br/> }]<br/> };<br/><br/> return
(<br/> <div className="relative w-full h-[600px]"><br/> <ReactECharts option={option}
style={{ height: '100%' }}<br/> onEvents={{ 'click': (e) => e.dataType === 'node'<br/>
&& setSelectedNode(e.data) }} /><br/> {selectedNode && (<br/> <NodeDetailPanel
node={selectedNode} onClose={() => setSelectedNode(null)} /><br/> )}<br/> </div><br/>
);<br/>}<br/><br/>function NodeDetailPanel({ node, onClose }) {<br/> return (<br/>
<div className="absolute top-4 right-4 bg-white rounded-lg shadow-lg p-4 w-64"><br/>
<button onClick={onClose} className="float-right">×</button><br/> <h3
className="font-bold">{node.type} #{node.entity_id}</h3><br/> <p className="text-sm
text-slate-600">Type: {node.type}</p><br/> <p className="text-sm
text-slate-600">Centrality: {node.centrality?.toFixed(3)}</p><br/> <p
className="text-sm text-slate-600">Community: {node.community}</p><br/> <button
className="mt-2 text-xs text-teal-600"><br/> View full profile →<br/> </button><br/>
</div><br/> );<br/>}
```

Verify: Dashboard "Network" tab renders the graph. Nodes are colored by type. Clicking a node shows the detail panel.

Time: 2 hours

Step 4.6 — Commit & Tag [INT]

```
git commit -m "Day 4: network pipeline (graph, Louvain, MO cluster) + ECharts
visualization"<br/>git tag -a v0.4-day4 -m "Day 4: network link analysis
working"<br/>git push origin main --tags
```

Time: 10 minutes

Day 5 — Predictive & Anomaly Pipeline (5 Tools)

Day 5 builds the third and final pipeline: the AI-driven predictive and anomaly detection capabilities that move the SCRB from reactive to proactive. This is the day that delivers the "Strategic Intelligence Hub" story from the problem statement. The goal is that an analyst can see a district risk choropleth (green to red), click a district to see the feature importance breakdown, view a 30-day forecast with confidence intervals, and see anomaly callouts when incidents deviate from behavioral patterns. The Zia AutoML model training is the longest single task of the sprint, so start it first thing in the morning and build the rest of the pipeline while it trains.

Day 5 success criteria

By end of Day 5: (1) Zia AutoML risk-scoring model is trained with documented accuracy, (2) risk_score(district) returns 0-100 with feature importance, (3) choropleth map renders with green→red gradient, (4) forecast(category) returns 30-day time series with 95% CI, (5) anomaly_detect() flags at least 3 anomalies in the synthetic data, (6) all 15 tools (5 viz + 5 network + 5 predictive) registered with orchestrator.

Step 5.1 — Train Zia AutoML Risk Scoring Model [BE]

Objective: Train a regression model that predicts crime count for a district in the next 30 days.

Files to create: scripts/train_automl.py

Commands:

```
# scripts/train_automl.py<br/>"""Train Zia AutoML model for district crime risk
forecasting."""<br/>import zcatalyst_sdk as catalyst<br/>import pandas as
pd<br/><br/>def prepare_training_data():<br/> """Build a training dataset from Data
Store.<br/><br/> Target: crime_count_next_30d (per district per date)<br/>
Features:<br/> - crime_count_7d, 14d, 30d (lag features)<br/> -
crime_count_same_period_last_year (seasonal)<br/> - district literacy_rate,
unemployment_rate, urbanization_rate<br/> - day_of_week, month,
is_festival_period<br/> """<br/> ds = catalyst.datastore()<br/><br/> # Pull 3 years of
daily crime counts per district<br/> sql = """<br/> SELECT d.district_id,
d.district_name,<br/> f.fir_date::date AS date,<br/> COUNT(*) AS crime_count,<br/>
d.urbanization_rate,<br/> sei.literacy_rate, sei.unemployment_rate,<br/>
sei.migration_index, sei.gdp_per_capita<br/> FROM fir f<br/> JOIN police_station ps ON
f.ps_id = ps.ps_id<br/> JOIN district d ON ps.district_id = d.district_id<br/> LEFT
JOIN socio_economic_indicator sei<br/> ON d.district_id = sei.district_id<br/> AND
EXTRACT(YEAR FROM f.fir_date) = sei.year<br/> WHERE f.fir_date >= '2022-01-01'<br/>
GROUP BY d.district_id, d.district_name, f.fir_date,<br/> d.urbanization_rate,
sei.literacy_rate,<br/> sei.unemployment_rate, sei.migration_index,
sei.gdp_per_capita<br/> ORDER BY d.district_id, date<br/> """<br/> rows =
ds.execute_query(sql)<br/> df = pd.DataFrame(rows)<br/> df['date'] =
pd.to_datetime(df['date'])<br/><br/> # Build lag features<br/> df =
df.sort_values(['district_id', 'date'])<br/> df['crime_count_7d'] =
df.groupby('district_id')['crime_count'] \<br/> .rolling(7).sum().reset_index(0,
drop=True)<br/> df['crime_count_14d'] = df.groupby('district_id')['crime_count']
\<br/> .rolling(14).sum().reset_index(0, drop=True)<br/> df['crime_count_30d'] =
df.groupby('district_id')['crime_count'] \<br/> .rolling(30).sum().reset_index(0,
drop=True)<br/><br/> # Target: next 30 days crime count<br/> df['target'] =
df.groupby('district_id')['crime_count'] \<br/>
.rolling(30).sum().shift(-30).reset_index(0, drop=True)<br/><br/> # Time features<br/>
```

```
df['day_of_week'] = df['date'].dt.dayofweek<br/> df['month'] =
df['date'].dt.month<br/> df['is_festival'] = df['month'].isin([10, 11, 8]).astype(int)
# Diwali, etc.<br/><br/> # Drop rows with NaN (first 30 and last 30 per district)<br/>
df = df.dropna()<br/><br/> return df[['crime_count_7d', 'crime_count_14d',
'crime_count_30d',<br/> 'urbanization_rate', 'literacy_rate',
'unemployment_rate',<br/> 'migration_index', 'gdp_per_capita',<br/> 'day_of_week',
'month', 'is_festival', 'target',<br/> 'district_id', 'date']]<br/><br/>def
train_model():<br/> df = prepare_training_data()<br/> print(f"Training data: {len(df)}
rows, {df['district_id'].nunique()} districts")<br/><br/> # Split: train on 2022-2024,
test on 2025<br/> train = df[df['date'] < '2025-01-01']<br/> test = df[df['date'] >=
'2025-01-01']<br/> print(f"Train: {len(train)}, Test: {len(test)}")<br/><br/> # Upload
to Catalyst and trigger Zia AutoML<br/> automl = catalyst.zia.automl()<br/><br/> job =
automl.create_job(<br/> name="ksp_district_risk_v1",<br/>
train_data=train.drop(['district_id', 'date'], axis=1),<br/>
target_column="target",<br/> problem_type="regression",<br/>
evaluation_metric="mae",<br/> time_limit_minutes=60, # 1 hour training<br/>
)<br/><br/> print(f"AutoML job started: {job.id}")<br/> print(f"Monitor: catalyst
zia:automl:status {job.id}")<br/><br/> # Block until done (or run async and check
later)<br/> result = job.wait_for_completion()<br/> print(f"Model trained! MAE:
{result.metrics['mae']:.2f}")<br/> print(f"Model ID: {result.model_id}")<br/><br/>
return result.model_id<br/><br/>if __name__ == "__main__":<br/> model_id =
train_model()<br/> # Save model_id for the risk_score tool to use<br/> with
open(".automl_model_id", "w") as f:<br/> f.write(model_id)
```

Run: python3 scripts/train_automl.py — this takes 60-90 minutes. Start it at 9 AM and build Steps 5.2-5.5 while it runs.

Verify: Script prints Model ID: ... and MAE: X.XX. MAE should be under 5 (meaning average prediction error of ~5 crimes per 30-day window).

Time: 90 minutes (mostly waiting for AutoML)

Step 5.2 — Implement risk_score Tool [BE]

Objective: Use the trained model to predict 30-day risk for a district.

Files to create: functions/predictive/risk_score.py

```
# functions/predictive/risk_score.py<br/>"""AI-driven district risk scoring using Zia
AutoML model."""<br/>from typing import Any, Dict<br/>import zcatalyst_sdk as
catalyst<br/><br/>MODEL_ID = open(".automl_model_id").read().strip()<br/><br/>def
risk_score(district: str, crime_type: str = "all",<br/> horizon_days: int = 30) ->
Dict[str, Any]:<br/> """Predict crime risk for a district.<br/><br/> Args:<br/>
district: district name<br/> crime_type: optional filter (default "all")<br/>
horizon_days: forecast horizon (default 30)<br/> Returns:<br/> {"risk_score": 0-100,
"confidence_interval": [...],<br/> "feature_importance": [...]}<br/> """<br/> ds =
catalyst.datastore()<br/> automl = catalyst.zia.automl()<br/><br/> # Fetch current
feature values for this district<br/> feat_sql = f"""<br/> SELECT
d.urbanization_rate,<br/> sei.literacy_rate, sei.unemployment_rate,<br/>
sei.migration_index, sei.gdp_per_capita,<br/> (SELECT COUNT(*) FROM fir f<br/> JOIN
police_station ps ON f.ps_id = ps.ps_id<br/> WHERE ps.district_id = d.district_id<br/>
AND f.fir_date >= CURRENT_DATE - INTERVAL '7 days') AS crime_7d,<br/> (SELECT COUNT(*)
FROM fir f<br/> JOIN police_station ps ON f.ps_id = ps.ps_id<br/> WHERE ps.district_id
= d.district_id<br/> AND f.fir_date >= CURRENT_DATE - INTERVAL '14 days') AS
```

```
crime_14d,<br/> (SELECT COUNT(*) FROM fir f<br/> JOIN police_station ps ON f.ps_id =
ps.ps_id<br/> WHERE ps.district_id = d.district_id<br/> AND f.fir_date >= CURRENT_DATE
- INTERVAL '30 days') AS crime_30d<br/> FROM district d<br/> LEFT JOIN
socio_economic_indicator sei<br/> ON d.district_id = sei.district_id<br/> AND sei.year
= EXTRACT(YEAR FROM CURRENT_DATE)<br/> WHERE d.district_name ILIKE '%{district}%'<br/>
LIMIT 1<br/> """<br/> features = ds.execute_query(feat_sql)[0]<br/><br/> # Add time
features<br/> import datetime<br/> now = datetime.datetime.now()<br/>
features['day_of_week'] = now.weekday()<br/> features['month'] = now.month<br/>
features['is_festival'] = 1 if now.month in [10, 11, 8] else 0<br/><br/> # Predict
using AutoML model<br/> prediction = automl.predict(MODEL_ID, features)<br/><br/> #
Convert predicted crime count to 0-100 risk score<br/> # (normalize against state-wide
distribution)<br/> predicted_count = prediction['value']<br/> risk = min(100, max(0,
(predicted_count / 50) * 100)) # 50 crimes = 100 risk<br/><br/> return {<br/>
"district": district,<br/> "risk_score": round(risk, 1),<br/> "predicted_crime_count":
round(predicted_count, 1),<br/> "confidence_interval": [<br/>
round(prediction['lower_ci'], 1),<br/> round(prediction['upper_ci'], 1)<br/> ],<br/>
"feature_importance": prediction.get('feature_importance', []),<br/> "model_version":
MODEL_ID,<br/> "horizon_days": horizon_days,<br/> "_data_refs": {<br/> "tables":
["district", "socio_economic_indicator", "fir"],<br/> "row_count": 1,<br/>
"query_sql": feat_sql,<br/> "model_version": MODEL_ID,<br/> "cache_hit": False,<br/>
}<br/> }
```

Verify: risk_score("Bengaluru Urban") returns a score 0-100 with feature importance showing which features drove the prediction.

Time: 1 hour

Step 5.3 — Implement forecast + socio_correlation [BE]

Objective: Time-series forecasting and socio-economic correlation analysis.

Files to create: functions/predictive/forecast.py, functions/predictive/socio_correlation.py

Code (forecast.py — simpler approach using moving averages + seasonal if AutoML forecast unavailable):

```
# functions/predictive/forecast.py<br/>"""30-day crime forecast with confidence
intervals."""<br/>from typing import Any, Dict, List<br/>from datetime import
datetime, timedelta<br/>import zcatalyst_sdk as catalyst<br/>import numpy as
np<br/><br/>def forecast(district: str, crime_type: str = "all",<br/> horizon_days:
int = 30) -> Dict[str, Any]:<br/> """Forecast daily crime count for the next
horizon_days.<br/><br/> Uses historical seasonal pattern + recent trend.<br/> For
production, replace with Zia AutoML time-series model.<br/> """<br/> ds =
catalyst.datastore()<br/><br/> # Fetch 2 years of daily counts for seasonal
pattern<br/> sql = f"""<br/> SELECT f.fir_date::date AS date, COUNT(*) AS count<br/>
FROM fir f<br/> JOIN police_station ps ON f.ps_id = ps.ps_id<br/> JOIN district d ON
ps.district_id = d.district_id<br/> WHERE d.district_name ILIKE '%{district}%'<br/>
AND f.fir_date >= CURRENT_DATE - INTERVAL '730 days'<br/> GROUP BY date ORDER BY
date<br/> """<br/> rows = ds.execute_query(sql)<br/> if len(rows) < 60:<br/> return
{"forecast": [], "error": "Insufficient history"}<br/><br/> counts =
np.array([r['count'] for r in rows])<br/><br/> # Decompose: trend (30-day moving avg) +
seasonal (day-of-week pattern)<br/> trend = np.convolve(counts, np.ones(30)/30,
mode='valid')<br/> dow_pattern = np.zeros(7)<br/> for i, c in enumerate(counts):<br/>
# Approximate day of week from index (assuming rows are daily)<br/> dow_pattern[i % 7]
+= c<br/> dow_pattern = dow_pattern / (len(counts) / 7)<br/><br/> # Project
forward<br/> last_trend = trend[-1]<br/> forecast_list = []<br/> for i in
```

```
range(horizon_days):<br/> dow = (len(counts) + i) % 7<br/> seasonal = dow_pattern[dow] - np.mean(dow_pattern)<br/> predicted = max(0, last_trend + seasonal)<br/> # Confidence interval: ±20% of predicted<br/> ci_width = predicted * 0.2<br/> forecast_list.append({<br/> "date": (datetime.now().date() + timedelta(days=i+1)).isoformat(),<br/> "predicted": round(predicted, 1),<br/> "lower_ci": round(max(0, predicted - ci_width), 1),<br/> "upper_ci": round(predicted + ci_width, 1),<br/> })<br/><br/> return {<br/> "district": district,<br/> "crime_type": crime_type,<br/> "forecast": forecast_list,<br/> "horizon_days": horizon_days,<br/> "method": "seasonal_naive_with_trend",<br/> "_data_refs": {<br/> "tables": ["fir", "police_station", "district"],<br/> "row_count": len(rows),<br/> "query_sql": sql,<br/> "cache_hit": False,<br/> }<br/> }
```

Code (socio_correlation.py):

```
# functions/predictive/socio_correlation.py<br/>"""Correlate crime patterns with socio-economic indicators."""<br/>import zcatalyst_sdk as catalyst<br/>import numpy as np<br/>from scipy import stats # pip install scipy<br/><br/>def socio_correlation(indicator: str = "all",<br/> crime_type: str = "all") -> Dict[str, Any]:<br/> """Compute Pearson correlation between crime and socio-economic indicators.<br/><br/> Args:<br/> indicator: literacy_rate, unemployment_rate, urbanization_rate, etc.<br/> crime_type: filter by crime type or "all"<br/> Returns:<br/> {"correlations": [{"indicator": str, "crime_type": str,<br/> "correlation": float, "p_value": float}]}<br/> """<br/> ds = catalyst.datastore()<br/> sql = """<br/> SELECT d.district_name, ct.category AS crime_type,<br/> d.urbanization_rate,<br/> sei.literacy_rate, sei.unemployment_rate,<br/> sei.migration_index, sei.gdp_per_capita,<br/> COUNT(f.fir_id) AS crime_count<br/> FROM fir f<br/> JOIN police_station ps ON f.ps_id = ps.ps_id<br/> JOIN district d ON ps.district_id = d.district_id<br/> JOIN crime_type ct ON f.crime_type_id = ct.crime_type_id<br/> LEFT JOIN socio_economic_indicator sei<br/> ON d.district_id = sei.district_id<br/> AND EXTRACT(YEAR FROM f.fir_date) = sei.year<br/> WHERE f.fir_date >= '2024-01-01'<br/> GROUP BY d.district_name, ct.category, d.urbanization_rate,<br/> sei.literacy_rate, sei.unemployment_rate,<br/> sei.migration_index, sei.gdp_per_capita<br/> """<br/> rows = ds.execute_query(sql)<br/><br/> indicators = ['literacy_rate', 'unemployment_rate',<br/> 'urbanization_rate', 'migration_index', 'gdp_per_capita']<br/> crime_types = list(set(r['crime_type'] for r in rows))<br/><br/> correlations = []<br/> for ind in indicators:<br/> for ct in crime_types:<br/> subset = [r for r in rows if r['crime_type'] == ct<br/> and r[ind] is not None]<br/> if len(subset) < 5:<br/> continue<br/> x = np.array([r[ind] for r in subset])<br/> y = np.array([r['crime_count'] for r in subset])<br/> r_val, p_val = stats.pearsonr(x, y)<br/> correlations.append({<br/> "indicator": ind,<br/> "crime_type": ct,<br/> "correlation": round(r_val, 3),<br/> "p_value": round(p_val, 4),<br/> "n": len(subset)<br/> })<br/><br/> correlations.sort(key=lambda c: abs(c['correlation']), reverse=True)<br/> return {"correlations": correlations[:20]} # top 20 strongest
```

Add to requirements.txt: scipy>=1.11, numpy>=1.24

Verify: forecast("Bengaluru Urban") returns 30 daily predictions. socio_correlation() returns correlations — theft should correlate positively with unemployment (in synthetic data).

Time: 2 hours

Step 5.4 — Implement anomaly_detect [BE]

Objective: Flag incidents that deviate from standard behavioral patterns using isolation forest.

Files to create: functions/predictive/anomaly_detect.py

```
# functions/predictive/anomaly_detect.py<br/>"""Anomaly detection using isolation
forest."""<br/>from typing import Any, Dict, List<br/>import zcatalyst_sdk as
catalyst<br/>from sklearn.ensemble import IsolationForest<br/>import numpy as
np<br/><br/>def anomaly_detect(district: str = "all",<br/> crime_type: str =
"all",<br/> window_days: int = 7) -> Dict[str, Any]:<br/> """Detect anomalous crime
patterns in the last window_days.<br/><br/> Features considered:<br/> - daily crime
count<br/> - crime type mix (entropy)<br/> - geographic spread (std dev of
lat/lng)<br/> - time-of-day distribution<br/><br/> Returns list of anomalies with
explanation.<br/> """<br/> ds = catalyst.datastore()<br/><br/> sql = f"""<br/> SELECT
f.fir_date::date AS date,<br/> f.crime_type_id,<br/> ct.category AS crime_type,<br/>
f.lat, f.lng,<br/> EXTRACT(HOUR FROM f.fir_time) AS hour<br/> FROM fir f<br/> JOIN
crime_type ct ON f.crime_type_id = ct.crime_type_id<br/> JOIN police_station ps ON
f.ps_id = ps.ps_id<br/> JOIN district d ON ps.district_id = d.district_id<br/> WHERE
f.fir_date >= CURRENT_DATE - INTERVAL '{window_days + 90} days'<br/> {'AND
d.district_name ILIKE %(district)s' if district != 'all' else ''}<br/> """<br/> rows =
ds.execute_query(sql, params={"district": district})<br/><br/> # Build daily feature
vectors for the last 90 days<br/> from collections import defaultdict<br/> daily =
defaultdict(lambda: {"count": 0, "types": set(),<br/> "lats": [], "lngs": [], "hours":
[]})<br/> for r in rows:<br/> daily[r['date']]["count"] += 1<br/>
daily[r['date']]["types"].add(r['crime_type'])<br/>
daily[r['date']]["lats"].append(r['lat'] or 0)<br/>
daily[r['date']]["lngs"].append(r['lng'] or 0)<br/>
daily[r['date']]["hours"].append(r['hour'] or 0)<br/><br/> # Build feature matrix<br/>
features = []<br/> dates = []<br/> for date, d in sorted(daily.items()):<br/>
features.append([<br/> d["count"],<br/> len(d["types"]), # type diversity<br/>
np.std(d["lats"]) if len(d["lats"]) > 1 else 0,<br/> np.std(d["lngs"]) if
len(d["lngs"]) > 1 else 0,<br/> np.std(d["hours"]) if len(d["hours"]) > 1 else 0,<br/>
])<br/> dates.append(date)<br/><br/> if len(features) < 30:<br/> return {"anomalies":
[], "error": "Insufficient data"}<br/><br/> X = np.array(features)<br/> # Fit
isolation forest on all but last window_days<br/> clf =
IsolationForest(contamination=0.1, random_state=42)<br/>
clf.fit(X[:-window_days])<br/><br/> # Predict on last window_days<br/> preds =
clf.predict(X[-window_days:])<br/> scores =
clf.decision_function(X[-window_days:])<br/><br/> anomalies = []<br/> for i, (pred,
score) in enumerate(zip(preds, scores)):<br/> if pred == -1: # anomaly<br/> date =
dates[-window_days + i]<br/> day_data = daily[date]<br/> anomalies.append({<br/>
"date": str(date),<br/> "anomaly_score": round(-score, 3), # higher = more
anomalous<br/> "features": {<br/> "crime_count": day_data["count"],<br/>
"type_diversity": len(day_data["types"]),<br/> "geo_spread":
round(np.std(day_data["lats"]), 4),<br/> },<br/> "explanation":
_explain_anomaly(day_data, X[:-window_days].mean(axis=0))<br/> })<br/><br/> return
{<br/> "anomalies": anomalies,<br/> "window_days": window_days,<br/> "model":
"isolation_forest_v1",<br/> "_data_refs": {<br/> "tables": ["fir", "crime_type",
"police_station", "district"],<br/> "row_count": len(rows),<br/> "query_sql":
sql,<br/> }<br/> }<br/><br/>def _explain_anomaly(day_data, baseline):<br/> """Generate
human-readable explanation of why a day is anomalous."""<br/> explanations = []<br/> if
day_data["count"] > baseline[0] * 1.5:<br/> explanations.append(f"Crime count
{day_data['count']} is 50%+ above baseline {baseline[0]:.0f}")<br/> if
len(day_data["types"]) > baseline[1] * 1.3:<br/> explanations.append(f"Unusual type
diversity ({len(day_data['types'])} types)")<br/> return "; ".join(explanations) or
"Statistical outlier in behavioral pattern"
```

Verify: anomaly_detect("Bengaluru Urban") returns at least 1 anomaly with an explanation. If none, modify test data to spike a day, re-run.

Time: 1.5 hours

Step 5.5 — Register Predictive Tools + Build Risk Choropleth [BE+FE]

Objective: Wire the 5 predictive tools into the orchestrator and build the risk choropleth map.

Backend changes:

```
# In router_agent.py:<br/>from functions.predictive.risk_score import risk_score as
_risk_score<br/>from functions.predictive.forecast import forecast as
_forecast<br/>from functions.predictive.socio_correlation import socio_correlation as
_socio_correlation<br/>from functions.predictive.anomaly_detect import anomaly_detect
as _anomaly_detect<br/>from functions.predictive.feature_importance import
get_feature_importance as _feature_importance<br/><br/># Add to build_tools() — 5
predictive tools<br/># Update ROUTER_SYSTEM_PROMPT — add 5 predictive tools to
AVAILABLE TOOLS<br/># Redeploy orchestrator
```

Frontend file: web-client/src/components/RiskChoropleth.tsx

```
// Maps risk_score output to a green→red choropleth on Karnataka districts<br/>// Uses
Mapbox fill-color interpolation:<br/>// 'fill-color': ['interpolate',
['linear'],<br/>// ['get', 'risk_score'],<br/>// 0, '#4daf4a', 25, '#ffff33', 50,
'#ff7f00', 75, '#e41a1c']<br/><br/>// Click a district → show feature importance
panel:<br/>// "Risk Score: 78/100<br/>// Driving factors: unemployment_rate (+28%),
recent_crime_7d (+22%),<br/>// festival_period (+18%), literacy_rate (-12%)"
```

Verify: Dashboard "Predictive" tab shows Karnataka choropleth. Click Bengaluru → feature importance panel shows top 4 drivers.

Time: 2 hours

Step 5.6 — Commit & Tag [INT]

```
git commit -m "Day 5: predictive pipeline (risk, forecast, anomaly, socio-corr) +
choropleth"<br/>git tag -a v0.5-day5 -m "Day 5: all 3 pipelines complete (15
tools)"<br/>git push origin main --tags
```

Time: 10 minutes

Day 6 — Explainability, Data Lineage, Polish

Day 6 ties the three pipelines together with the explainability framework and polishes the dashboard for demo. The goal is that every dashboard cell and every AI prediction has a "Why this?" panel showing the full data lineage, and that the dashboard looks professional enough to present to the jury. This is also the day to wire up the PDF export feature (SmartBrowz) so analysts can download intelligence briefings.

Day 6 success criteria

By end of Day 6: (1) Every dashboard cell has a "Why this?" expandable panel, (2) the panel shows source FIRs, SQL, model version, confidence, (3) PDF export produces a downloadable intelligence briefing with embedded visualizations, (4) all 4 role-based views (investigator/analyst/SP/IGP) render correctly with PII masking, (5) dashboard looks visually polished (consistent colors, spacing, typography).

Step 6.1 — Build Lineage Viewer Component [FE]

Objective: Render the evidence trail / data lineage as an expandable "Why this?" panel.

Files to create: web-client/src/components/LineageViewer.tsx

```
// web-client/src/components/LineageViewer.tsx<br/>import { useState } from
'react';<br/><br/>export function LineageViewer({ trailId }) {<br/> const [expanded,
setExpanded] = useState(false);<br/> const [trail, setTrail] =
useState(null);<br/><br/> const loadTrail = async () => {<br/> if (!expanded && !trail)
{<br/> const res = await fetch(`/api/audit/${trailId}`);<br/> setTrail(await
res.json());<br/> }<br/> setExpanded(!expanded);<br/> };<br/><br/> return (<br/> <div
className="border-l-2 border-teal-500 pl-3 my-2"><br/> <button
onClick={loadTrail}<br/> className="text-xs text-teal-600 hover:underline"><br/>
{expanded ? '▼' : '▶'} Why this answer?<br/> </button><br/> {expanded && trail &&
(<br/> <div className="mt-2 p-3 bg-slate-50 rounded text-xs space-y-2"><br/>
<div><br/> <strong>Agents invoked:</strong> {trail.agents_invoked.join(', ')}<br/>
</div><br/> <div><br/> <strong>Confidence:</strong><br/> <div className="inline-block
w-32 h-2 bg-slate-200 rounded ml-2"><br/> <div className="h-full bg-teal-500
rounded"<br/> style={{width: `${trail.confidence * 100}%`}} /><br/> </div><br/> <span
className="ml-2">{(trail.confidence * 100).toFixed(0)}%</span><br/> </div><br/>
<div><br/> <strong>Tool calls ({trail.tool_calls.length}):</strong><br/> <ul
className="ml-4 list-disc"><br/> {trail.tool_calls.map((tc, i) => (<br/> <li
key={i}><br/> <code>{tc.tool}({JSON.stringify(tc.params).slice(0, 60)}...)</code><br/>
<span className="text-slate-500 ml-2"><br/> ({tc.latency_ms}ms{tc.cache_hit ? ',
cached' : ''})<br/> </span><br/> </li><br/> ))}<br/> </ul><br/> </div><br/> <div><br/>
<strong>Data references:</strong><br/> <ul className="ml-4 list-disc"><br/>
<li>Tables: {trail.data_references.tables_touched.join(', ')}</li><br/> <li>Rows
touched: {trail.data_references.total_rows}</li><br/>
{trail.data_references.queries_sql?.[0] && (<br/> <li><br/> <details><br/> <summary
className="cursor-pointer">View SQL ({trail.data_references.queries_sql.length}
queries)</summary><br/> <pre className="mt-1 p-2 bg-white overflow-x-auto
text-[10px]"><br/> {trail.data_references.queries_sql[0]}<br/> </pre><br/>
</details><br/> </li><br/> )}<br/> </ul><br/> </div><br/> {trail.model_info && (<br/>
<div><br/> <strong>Model:</strong> {trail.model_info.model_type}<br/>
v{trail.model_info.model_version}<br/> <span className="text-slate-500 ml-2"><br/>
(trained: {trail.model_info.training_window})<br/> </span><br/> </div><br/> )}<br/>
{trail.reasoning_steps?.length > 0 && (<br/> <div><br/> <strong>Reasoning
```

```
steps:</strong><br/> <ol className="ml-4 list-decimal"><br/>
{trail.reasoning_steps.slice(0, 5).map((s, i) => (<br/> <li key={i}
className="text-slate-600">{s.slice(0, 120)}...</li><br/> ))}<br/> </ol><br/>
</div><br/> )}<br/> <div className="pt-2 border-t border-slate-200
text-slate-500"><br/> Audit ID: <code>{trail.audit_id || trail.trail_id}</code><br/>
</div><br/> </div><br/> )}<br/> </div><br/> );<br/>}
```

Verify: Click "Why this?" on any dashboard cell → panel expands showing tool calls, SQL, model version, confidence.

Time: 1.5 hours

Step 6.2 — Implement PDF Export via SmartBrowz [BE]

Objective: Generate a downloadable PDF intelligence briefing from the current dashboard state.

Files to create: functions/pdf_export/handler.py, functions/pdf_export/template.html

```
# functions/pdf_export/handler.py<br/>"""Generate PDF intelligence briefing via
SmartBrowz headless rendering."""<br/>import zcatalyst_sdk as catalyst<br/>from
datetime import datetime<br/><br/>def handler(event, context):<br/> """Generate PDF
from dashboard state.<br/><br/> event: {<br/> "user_id": "...",<br/>
"dashboard_state": {<br/> "district": "Bengaluru Urban",<br/> "crime_type":
"Theft",<br/> "date_range": "2025-01-01 to 2025-06-30",<br/> "visualizations": [...],
# snapshot data<br/> "evidence_trails": [...]<br/> }<br/> }<br/> """<br/> smartbrowz =
catalyst.smartbrowz()<br/><br/> # Render HTML template with dashboard state
injected<br/> html = _render_template(event["dashboard_state"])<br/><br/> # SmartBrowz
renders HTML to PDF<br/> pdf_bytes = smartbrowz.render_pdf(<br/>
html_content=html,<br/> options={<br/> "format": "A4",<br/> "print_background":
True,<br/> "margin": {"top": "20mm", "bottom": "20mm",<br/> "left": "15mm", "right":
"15mm"}<br/> }<br/> )<br/><br/> # Upload to Stratus<br/> filename =
f"briefing_{datetime.now().strftime('%Y%m%d_%H%M%S')}.pdf"<br/> stratus =
catalyst.stratus()<br/> file_obj =
stratus.folder("intelligence_briefings").upload_file(<br/> filename, pdf_bytes<br/>
)<br/><br/> # Log to audit<br/>
catalyst.nosql().collection("audit_log").insert_row({<br/> "action":
"pdf_export",<br/> "user_id": event["user_id"],<br/> "filename": filename,<br/>
"timestamp": datetime.utcnow().isoformat(),<br/> "dashboard_state":
event["dashboard_state"]<br/> })<br/><br/> return {"download_url":
file_obj.download_url, "filename": filename}<br/><br/>def
_render_template(state):<br/> """Render HTML template with dashboard data."""<br/>
with open("functions/pdf_export/template.html") as f:<br/> tpl = f.read()<br/><br/>
return tpl.replace("{{DISTRICT}}", state.get("district", "Karnataka")) \<br/>
.replace("{{DATE_RANGE}}", state.get("date_range", "")) \<br/>
.replace("{{GENERATED_AT}}", datetime.now().strftime("%Y-%m-%d %H:%M")) \<br/>
.replace("{{VIS_DATA}}", json.dumps(state.get("visualizations", []))) \<br/>
.replace("{{TRAILS}}", json.dumps(state.get("evidence_trails", [])))
```

Files to create: functions/pdf_export/template.html — a professional HTML template with header (KSP logo, briefing title, date), body (visualizations as embedded images/data), footer (audit ID, page numbers).

Verify: Click "Export PDF" in dashboard → downloads briefing_YYYYMMDD_HHMMSS.pdf → opens with embedded visualizations and evidence trails.

Time: 2 hours

Step 6.3 — Implement Role-Based Views + PII Masking [BE+FE]

Objective: 4 roles see different dashboard views with appropriate PII masking.

Backend changes (in each tool function):

```
# Add RBAC check at the start of each tool:<br/>def _check_rbac(user, resource,
action="read"):<br/> """Enforce row-level security based on user role and
jurisdiction."""<br/> role = user["role"]<br/> jurisdiction =
user["jurisdiction"]<br/><br/> if role == "igp":<br/> # IGP sees aggregates only, no
PII<br/> return {"allow": True, "mask_pii": True, "aggregate_only": True}<br/> elif
role == "sp":<br/> # SP sees own district only<br/> return {"allow": "district_id" in
jurisdiction,<br/> "district_filter": jurisdiction.get("district_id"),<br/>
"mask_pii": False}<br/> elif role == "scrb_analyst":<br/> # Analyst sees everything,
no PII mask by default<br/> return {"allow": True, "mask_pii": False}<br/> elif role ==
"investigator":<br/> # Investigator sees own PS only, no PII mask for own cases<br/>
return {"allow": "ps_id" in jurisdiction,<br/> "ps_filter":
jurisdiction.get("ps_id"),<br/> "mask_pii": False}<br/> return {"allow":
False}<br/><br/># Apply PII masking in tool output:<br/>def _mask_pii(record,
mask=True):<br/> if not mask:<br/> return record<br/> if "name" in record:<br/>
record["name"] = record["name"][0] + "***" if record["name"] else ""<br/> if "address"
in record:<br/> record["address"] = "***"<br/> return record
```

Frontend: 4 dashboard variants — AnalystDashboard.tsx (full), SpDashboard.tsx (district-scoped), InvestigatorDashboard.tsx (PS-scoped), IgpDashboard.tsx (aggregates only, no PII).

Verify: Login as each of the 4 test users → dashboard shows different scope and PII masking. Investigator cannot see Bengaluru Rural data. IGP sees no names.

Time: 2 hours

Step 6.4 — Visual Polish [FE]

Objective: Make the dashboard look professional and demo-ready.

Checklist:

- Consistent color palette: teal #0e7c7b primary, amber #c4791b accent, navy #0f2740 text
- Typography: Inter for body, Playfair Display for headings (or system fonts if Inter unavailable)
- Spacing: 16px base unit, 24px between sections, 8px between elements
- Loading states: skeleton screens for all data fetches (not spinners)
- Error states: friendly error messages with retry buttons
- Responsive: works on 1280px+ screens (demo projector resolution)
- Empty states: "No data for this filter" with suggestions
- Tooltips on all chart elements
- Smooth transitions (200ms ease) on all interactions

Time: 2 hours

Step 6.5 — Commit & Tag [INT]

```
git commit -m "Day 6: lineage viewer, PDF export, RBAC views, visual polish"<br/>git
tag -a v0.6-day6 -m "Day 6: explainability + polish complete"<br/>git push origin main
--tags
```

Time: 10 minutes

Day 7 — Integration, Deployment, Demo

Day 7 is the day that determines whether you win or lose. The platform is built; today is about making it bulletproof for the demo and capturing the submission artifacts. The morning is for integration testing and bug fixing, the afternoon is for deployment and demo rehearsal, and the evening is for recording the 3-minute demo video and capturing screenshots. Do not add new features today; if something is broken, fix it, but do not start anything new.

Day 7 success criteria

By end of Day 7: (1) All 50 eval queries pass, (2) platform deployed to Catalyst production at a public URL, (3) 3-minute demo video recorded and uploaded, (4) 8-10 screenshots captured for submission PPTX, (5) GitHub repo is public with README, (6) submission PPTX filled and ready to submit.

Step 7.1 — Run Eval Suite [INT]

Objective: Run the 50-query eval suite and fix any failures.

Files to create: tests/eval_queries.json, tests/run_eval.py

```
# tests/eval_queries.json (first 5 of 50)<br/>[<br/> {<br/> "id": "viz_01",<br/>
"query": "Show me theft trends in Bengaluru Urban from January to June 2025",<br/>
"expected_agents": ["visualization"],<br/> "expected_tools": ["trend_query"],<br/>
"expected_min_result_keys": ["total_cases", "monthly"],<br/>
"expected_min_total_cases": 100<br/> },<br/> {<br/> "id": "viz_02",<br/> "query":
"Where are the crime hotspots in Mysuru district?",<br/> "expected_agents":
["visualization"],<br/> "expected_tools": ["hotspot_kde"],<br/>
"expected_min_result_keys": ["cells", "is_red_zone"]<br/> },<br/> {<br/> "id":
"net_01",<br/> "query": "Show me the criminal network around accused Ravi Kumar",<br/>
"expected_agents": ["network"],<br/> "expected_tools": ["lookup_accused",
"build_graph"],<br/> "expected_min_nodes": 5<br/> },<br/> {<br/> "id": "pred_01",<br/>
"query": "What is the 30-day crime risk forecast for Bengaluru Urban?",<br/>
"expected_agents": ["predictive"],<br/> "expected_tools": ["risk_score"],<br/>
"expected_risk_range": [0, 100]<br/> },<br/> {<br/> "id": "compound_01",<br/> "query":
"Show theft hotspots in Bengaluru North and the repeat offender network within
them",<br/> "expected_agents": ["visualization", "network"],<br/> "expected_tools":
["hotspot_kde", "build_graph", "find_clusters"]<br/> }<br/> # ... 45 more queries
covering all 15 tools, edge cases, RBAC denials<br/>]
```

```
# tests/run_eval.py<br/>import json, requests, sys<br/><br/>def run_eval():<br/> with
open("tests/eval_queries.json") as f:<br/> queries = json.load(f)<br/><br/> results =
[]<br/> for q in queries:<br/> print(f"Running {q['id']}...", end=" ")<br/> try:<br/>
resp = requests.post("https://your-catalyst-url/api/query", json={<br/> "query":
q["query"],<br/> "session_id": f"eval_{q['id']}",<br/> "user": {"id": "eval", "role":
"scrb_analyst", "jurisdiction": "state"}<br/> }, timeout=30)<br/> data =
resp.json()<br/><br/> # Validate response<br/> passed = _validate(data, q)<br/>
results.append({"id": q["id"], "passed": passed, "error": None if passed else
"validation failed"})<br/> print("PASS" if passed else "FAIL")<br/> except Exception
as e:<br/> results.append({"id": q["id"], "passed": False, "error": str(e)})<br/>
print(f"ERROR: {e}")<br/><br/> passed = sum(1 for r in results if r["passed"])<br/>
total = len(results)<br/> print(f"\n{passed}/{total} passed
({passed/total*100:.0f}%)")<br/><br/> if passed < total:<br/>
print("\nFailures:")<br/> for r in results:<br/> if not r["passed"]:<br/> print(f"
{r['id']}: {r['error']}")<br/><br/> return passed == total<br/><br/>def
```

```
_validate(data, q):<br/> if "error" in data:<br/> return False<br/> if
"agents_invoked" in q.get("expected_agents", []):<br/> if not
set(q["expected_agents"]).issubset(set(data.get("agents_invoked", []))):<br/> return
False<br/> # ... more validation<br/> return True<br/><br/>if __name__ ==
"__main__":<br/> success = run_eval()<br/> sys.exit(0 if success else 1)
```

Run: python3 tests/run_eval.py

Target: 48/50 passing (96%). Fix failures by debugging the specific tool or prompt. If a query is fundamentally broken, mark it as "known issue" in the eval file and move on.

Time: 2 hours

Step 7.2 — Deploy to Catalyst Production [BE+INT]

Objective: Deploy all functions, web client, and config to production Catalyst environment.

Commands:

```
# Switch to production environment<br/>catalyst env:set production<br/><br/># Deploy
all functions<br/>catalyst functions:deploy functions/orchestrator/ --name
orchestrator --env production<br/>catalyst functions:deploy functions/visualization/
--name viz_tools --env production<br/>catalyst functions:deploy functions/network/
--name network_tools --env production<br/>catalyst functions:deploy
functions/predictive/ --name pred_tools --env production<br/>catalyst functions:deploy
functions/pdf_export/ --name pdf_export --env production<br/>catalyst functions:deploy
functions/nightly_batch/ --name nightly --env production<br/><br/># Deploy web client
to Slate<br/>catalyst slate:deploy web-client/ --name ksp-dashboard --env
production<br/><br/># Configure API Gateway routes<br/>catalyst apigw:routes:import
config/routes.json --env production<br/><br/># Configure Cron jobs (4
nightly)<br/>catalyst cron:create --schedule "0 2 * * *" --function
nightly/hotspot_recompute --env production<br/>catalyst cron:create --schedule "0 3 *
* *" --function nightly/forecast_refresh --env production<br/>catalyst cron:create
--schedule "0 4 * * *" --function nightly/anomaly_scan --env production<br/>catalyst
cron:create --schedule "0 5 * * *" --function nightly/graph_refresh --env
production<br/><br/># Map custom domain (optional but professional)<br/>catalyst
domain:map ksp-analytics.your-domain.com --env production<br/><br/># Verify
deployment<br/>catalyst functions:list --env production<br/>catalyst slate:list --env
production<br/>curl https://ksp-analytics.your-domain.com/api/health # should return
{"status":"ok"}
```

Verify: Public URL loads the dashboard. Login works. A test query returns real data.

Time: 1.5 hours

Step 7.3 — Capture Screenshots [FE+INT]

Objective: Capture 8-10 high-quality screenshots for the submission PPTX (slide 11).

Screenshots to capture (at 1920x1080):

1. SCRB Strategic Dashboard — full Karnataka heatmap with red-zone pulsing
2. District Drill-Down — Bengaluru North zoomed in, PS-level hotspots

3. Network Graph Explorer — criminal network around a repeat offender, community hulls
4. Repeat Offender Profile — cross-jurisdictional timeline, MO evolution
5. Risk Choropleth — district risk scores (green→red) with feature importance panel
6. 30-Day Forecast — line chart with 95% confidence intervals
7. Anomaly Detection — flagged incidents with "Why anomalous?" panel
8. Lineage Viewer — full provenance for one dashboard cell (SQL, model version, confidence)
9. PDF Intelligence Briefing — sample exported PDF
10. Login + Role Selection — shows the 4-role RBAC

Tool: Use Chrome DevTools > Device Toolbar > Responsive > 1920x1080, then Cmd+Shift+S or use a screenshot extension like GoFullPage.

Save to: docs/screenshots/ with descriptive names.

Time: 1 hour

Step 7.4 — Record 3-Minute Demo Video [INT]

Objective: Record the demo video that the jury will watch.

Demo script (use this exactly — 180 seconds total):

Time	Section	What to Show	Narration (spoken)
0:00-0:20	Hook	Title slide + problem statement	"Every day, KSP officers drown in Excel sheets while hidden criminal networks go undetected. We built a platform that turns siloed data into strategic intelligence."
0:20-0:50	Architecture	Architecture diagram slide	"Three analytics pipelines on Zoho Catalyst: visualization, network, and predictive. Built on HuggingFace smolagents for explainable agent orchestration. Zero external dependencies."
0:50-2:20	Live Demo	Screen recording of dashboard	Walk through the 4 key scenarios (see below)
2:20-2:50	Explainability	Click "Why this?" on a prediction	"Every AI prediction carries full data lineage — source FIRs, SQL, model version, confidence. This is what accountable AI for law enforcement looks like."
2:50-3:00	Close	Team slide + URL	"Deployed at ksp-analytics.zoho.com. Open source on GitHub. Ready for pilot at KSP. Thank you."

Live demo scenarios (90 seconds, 30s each):

Scenario 1 (Visualization): Open dashboard → show Karnataka heatmap → click Bengaluru → drill to PS → point out red-zone pulsing

Scenario 2 (Network): Search accused "Ravi Kumar" → show network graph → click "Find Communities" → point out the organized crime cluster

Scenario 3 (Predictive): Click "Risk Forecast" → show choropleth → click Bengaluru → show feature importance → show 30-day forecast line

Recording tool: OBS Studio (free) or Loom. Record at 1920x1080, 30fps, MP4.

Upload to: YouTube (unlisted) or Vimeo. Get the shareable link for submission PPTX slide 13.

Time: 2 hours (including rehearsal and retakes)

Step 7.5 — Fill Submission PPTX + Final Push [INT]

Objective: Fill the submission template with all final content and links.

Use: The smolagents-updated PPTX from /home/z/my-project/download/KSP_Analytics_Datathon_2026_Submission_smolagents.pptx as the starting point.

Updates needed:

- Slide 1: Team name, leader name, team size
- Slide 11: Insert the 8-10 screenshots captured in Step 7.3
- Slide 12: Insert actual benchmark numbers from eval run
- Slide 13: Insert GitHub URL, demo video URL, deployed URL
- Slide 14: Confirm roadmap matches what was actually built

```
# Final commit and push<br/>git add -A<br/>git commit -m "Day 7: eval pass, production deploy, screenshots, demo video, submission"<br/>git tag -a v1.0-final -m "Submission ready"<br/>git push origin main --tags<br/><br/># Make repo public (if not already)<br/>gh repo edit ksp-crime-analytics-platform --visibility public<br/><br/># Final verification: open the submission PPTX, confirm all slides are filled<br/># Submit to Datathon portal
```

Time: 1 hour

Submission Checklist

Use this checklist on Day 7 evening before clicking submit. Every item must be checked. A submission missing any item will be penalized.

#	Item	Where	Done?
1	GitHub repo is PUBLIC (not private)	github.com/your-team/ksp-crime-analytics-platform	[ ]
2	README.md has setup instructions and deploy steps	GitHub repo root	[ ]
3	Demo video is 3 minutes (±15 seconds) and publicly viewable	YouTube/Vimeo link	[ ]
4	Deployed URL loads the dashboard (test in incognito)	ksp-analytics.zoho.com or custom domain	[ ]
5	Login works for all 4 test users (investigator/analyst/SP/IGP)	Deployed URL login page	[ ]
6	At least 3 demo queries return real, grounded answers	Live test on deployed URL	[ ]
7	Evidence trail / 'Why this?' panel works on at least one cell	Live test on deployed URL	[ ]
8	Submission PPTX has all 14 content slides filled	KSP_Analytics_Datathon_2026_Submission_smolagents.pptx	[ ]
9	Screenshots (slide 11) are actual screenshots, not placeholders	PPTX slide 11	[ ]
10	GitHub link, demo video link, deployed link all present (slide 13)	PPTX slide 13	[ ]
11	Team name, leader name, team size filled (slide 1)	PPTX slide 1	[ ]
12	Problem statement on slide 1 matches the Datathon topic	PPTX slide 1	[ ]
13	Architecture diagram on slide 7 is the smolagents-updated version	PPTX slide 7	[ ]
14	Catalyst services list on slide 9 includes smolagents	PPTX slide 9	[ ]
15	Cost estimate on slide 10 is realistic and matches blueprint	PPTX slide 10	[ ]
16	Performance numbers on slide 12 are actual (not aspirational)	PPTX slide 12	[ ]
17	Future roadmap on slide 14 mentions agent-lightning APO	PPTX slide 14	[ ]
18	Code is clean (no debug print statements, no hardcoded secrets)	GitHub repo	[ ]
19	.env.example exists, real .env is in .gitignore	GitHub repo	[ ]
20	Submission submitted before deadline (set a 2-hour buffer)	Datathon portal	[ ]

Appendix — Troubleshooting Guide

The most common issues you will hit during the 7-day sprint, with their fixes. Bookmark this section.

A.1 smolagents Issues

Issue: ImportError: No module named smolagents on Catalyst Functions

Fix: Ensure functions/orchestrator/requirements.txt exists and is picked up by the Catalyst CLI. Run catalyst functions:deploy --include-requirements. Verify the runtime is python3.12 (smolagents requires 3.10+).

Issue: agent.last_run_steps is empty or None after run()

Fix: This happens if the agent crashed before completing any steps. Check agent.last_error and Catalyst Functions logs. Common cause: LLM endpoint unreachable or API key invalid. Verify with curl first.

Issue: Agent produces code that throws NameError (tool not defined)

Fix: The system prompt lists tools that are not actually registered. Ensure every tool in ROUTER_SYSTEM_PROMPT is also in build_tools(). The names must match exactly.

A.2 Catalyst Data Store Issues

Issue: execute_query returns empty results

Fix: Check the SQL is valid PostgreSQL (Catalyst Data Store uses Postgres). Test the query in catalyst datastore:query "SELECT ..." first. Common cause: case sensitivity — column names are lowercase by default, use double quotes for exact match.

Issue: Bulk import fails with "column type mismatch"

Fix: CSV column order must match table column order. Check for NULL values in NOT NULL columns. For DATE columns, use YYYY-MM-DD format. For DECIMAL, use dot decimal separator.

A.3 QuickML / LLM Issues

Issue: QuickML endpoint returns 401 Unauthorized

Fix: Verify the API key is set as an environment variable (not hardcoded). Run catalyst functions:env:list orchestrator to confirm. QuickML keys are project-scoped — ensure you are using the key from the same project.

Issue: LLM response is too slow (>10 seconds per query)

Fix: (1) Reduce max_tokens in the LLM config. (2) Set temperature=0.1 for deterministic routing. (3) Cache common queries (implement a simple hash-based cache in the orchestrator). (4) Use a smaller model for intent classification, larger only for response synthesis.

A.4 Zia AutoML Issues

Issue: AutoML training job fails or hangs

Fix: (1) Ensure training data has at least 100 rows. (2) Check for NULL values in feature columns — AutoML does not handle them gracefully. (3) Reduce time_limit_minutes to 30 if the job times out. (4) If it still fails, fall back to the seasonal-naive forecast in forecast.py (Step 5.3) and note in the demo that AutoML is "in progress."

A.5 Mapbox / Frontend Issues

Issue: Map is blank or shows "Invalid API token"

Fix: Verify VITE_MAPBOX_TOKEN is set in .env.local and the token is valid at mapbox.com. Restart the dev server after changing env vars (npm run dev does not hot-reload env).

Issue: Heatmap layer does not appear

Fix: The GeoJSON source must be loaded before the layer. Ensure map.on('load', ...) wraps the addSource and addLayer calls. Check the browser console for errors — common cause: features array is empty because the API call failed.

A.6 Demo Day Failures

Issue: Live demo fails (network, LLM down, agent confusion)

Fix: (1) Pre-cache responses for all demo queries by running them 30 minutes before the demo and storing in Catalyst Cache. (2) Have the 3-minute demo video playing in a separate tab as fallback — if live demo fails, switch to the video. (3) Bring a personal hotspot in case venue WiFi is unreliable. (4) Rehearse the demo 5+ times so you can recover from any failure smoothly.

> **Final word**
>
> You have everything you need: the blueprint PDF, the smolagents addendum, the code skeleton, the updated PPTX, and this implementation guide. The only variable left is execution. Follow the steps in order, do not skip verification, commit often, and on Day 7 focus on polish and demo rehearsal rather than new features. The teams that win hackathons are not the ones with the most features; they are the ones whose demo works flawlessly and tells a compelling story. Build less, polish more, win. Good luck.